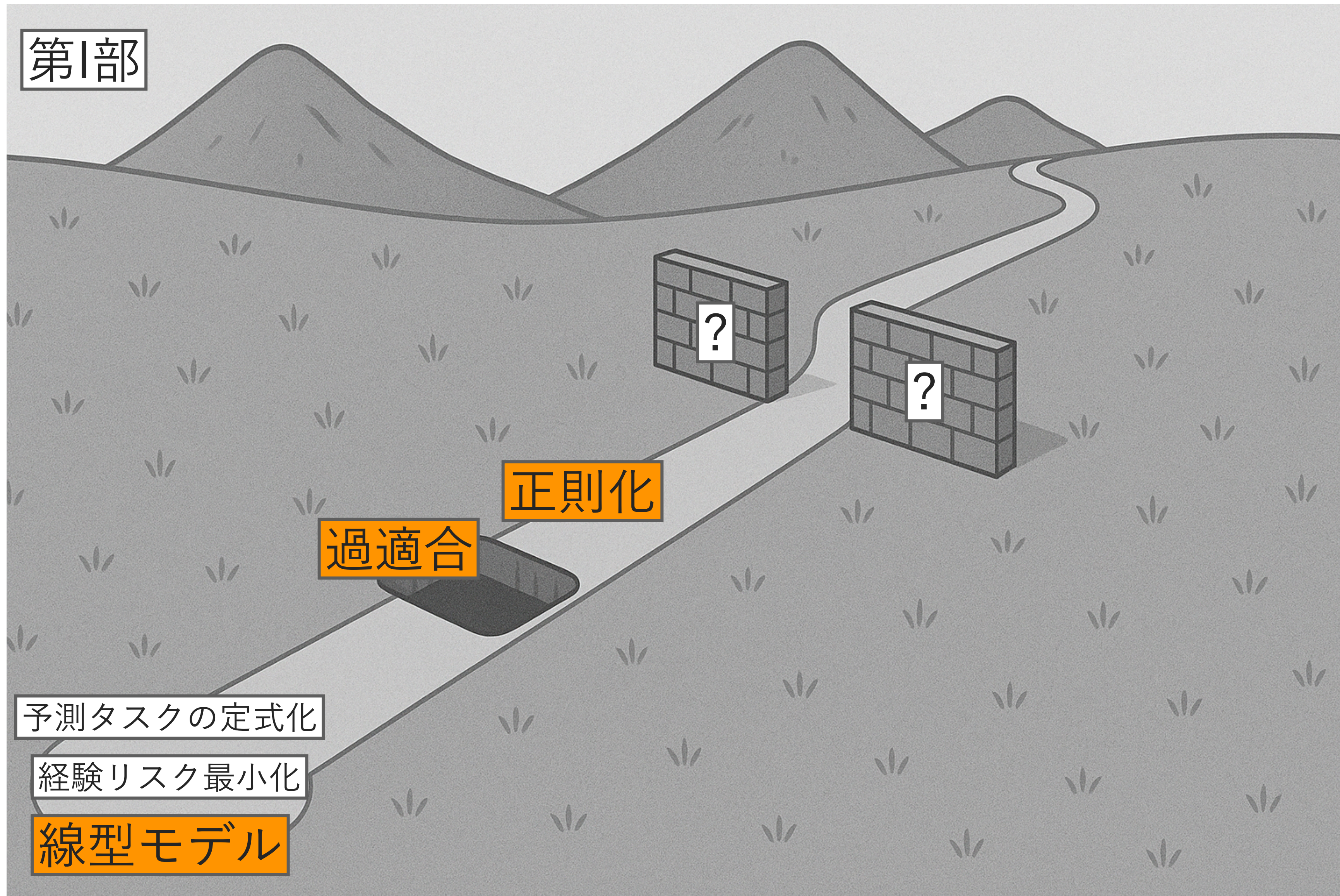


第3回

手嶋毅志 (Takeshi Teshima)

ロードマップ



質問と回答

告知

- シラバスの講義計画の調整を行います
 - 大枠は変わりません
- 小テストは月曜日に公開します



線型モデル

基本知識の確認＞ベクトルの内積

同じ位置の成分同士を掛けて、足し上げた値を内積（inner product）という

■ ベクトルの内積 $\langle \mathbf{a}, \mathbf{b} \rangle$

$$\mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_d \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_d \end{bmatrix} \text{ に対して, } \langle \mathbf{a}, \mathbf{b} \rangle := a_1 b_1 + \cdots + a_d b_d$$

■ 別の表記法 $\mathbf{a}^\top \mathbf{b}$ （ $1 \times d$ 行列と $d \times 1$ 行列の積とみなして計算）

$$\mathbf{a}^\top \mathbf{b} = [a_1 \ a_2 \ \cdots \ a_d] \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_d \end{bmatrix} = a_1 b_1 + \cdots + a_d b_d$$

【補足】 $\mathbf{a}^\top \mathbf{b}$ という表記は、内積の記号 $\langle \cdot, \cdot \rangle$ を定義するまでもなく伝わりが多いので、ある意味スマートな表記である。

基本知識の確認＞ベクトルの内積

内積の主な出番は3つ

- 重み付き和を簡潔に表す（成分ごとの重み $\mathbf{w} = (w_1, \dots, w_d)$ ）

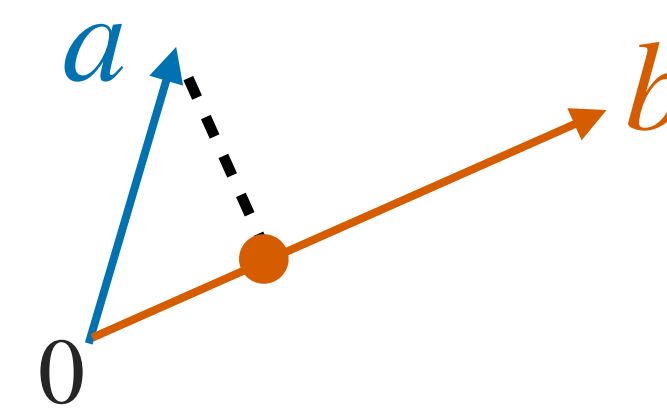
$$\mathbf{w}^T \mathbf{x} = w_1 x_1 + \dots + w_d x_d$$

- 2つのベクトルの向き of 揃い具合（同じ向きで最大，逆向きで最小）

$$\langle \nearrow, \searrow \rangle = -1, \quad \langle \nearrow, \nearrow \rangle = 0, \quad \langle \nearrow, \nwarrow \rangle = 1 \quad (\text{長さ1の場合})$$

- 特定の方向の成分の抽出（ベクトル $\mathbf{a}$ の「ベクトル $\mathbf{b}$ 方向成分」）

$$\mathbf{a} = \frac{\langle \mathbf{a}, \mathbf{b} \rangle}{\langle \mathbf{b}, \mathbf{b} \rangle} \mathbf{b} + (\mathbf{b} \text{ に直交する成分})$$



【補足】 同じ向きを向いている場合に内積が最も大きくなるという主張はコーシー・シュワルツの不等式から従う。ここでベクトルの長さは $\|\cdot\| = \sqrt{\langle \cdot, \cdot \rangle}$ で測る。

最後の $\mathbf{a} = (\langle \mathbf{a}, \mathbf{b} \rangle / \langle \mathbf{b}, \mathbf{b} \rangle) \mathbf{b} + (\mathbf{b} \text{ に直交する成分})$ という分解は、 $\langle \mathbf{a} - (\langle \mathbf{a}, \mathbf{b} \rangle / \langle \mathbf{b}, \mathbf{b} \rangle) \mathbf{b}, \mathbf{b} \rangle = \langle \mathbf{a}, \mathbf{b} \rangle - (\langle \mathbf{a}, \mathbf{b} \rangle / \langle \mathbf{b}, \mathbf{b} \rangle) \cdot \langle \mathbf{b}, \mathbf{b} \rangle = 0$ により確かめられる。

基本知識の確認＞ベクトルの内積の基本性質

内積は線型性と対称性をもつ

■ 線型性 (linearity) : $\langle \mathbf{x}, \alpha \mathbf{y} + \beta \mathbf{z} \rangle = \alpha \langle \mathbf{x}, \mathbf{y} \rangle + \beta \langle \mathbf{x}, \mathbf{z} \rangle$

- 一般に, 左側についても同時に線型 (双線型)

$$\langle a\mathbf{x} + b\mathbf{y}, c\mathbf{z} + d\mathbf{w} \rangle = ac\langle \mathbf{x}, \mathbf{z} \rangle + ad\langle \mathbf{x}, \mathbf{w} \rangle + bc\langle \mathbf{y}, \mathbf{z} \rangle + bd\langle \mathbf{y}, \mathbf{w} \rangle$$

- これは括弧の分配法則の一般化 $(a + b)(c + d) = ac + bd + bc + bd$
- 特に今回は「重みに関する線型性」という形で関係 : $(\mathbf{v} + \mathbf{w})^\top \mathbf{x} = \mathbf{v}^\top \mathbf{x} + \mathbf{w}^\top \mathbf{x}$

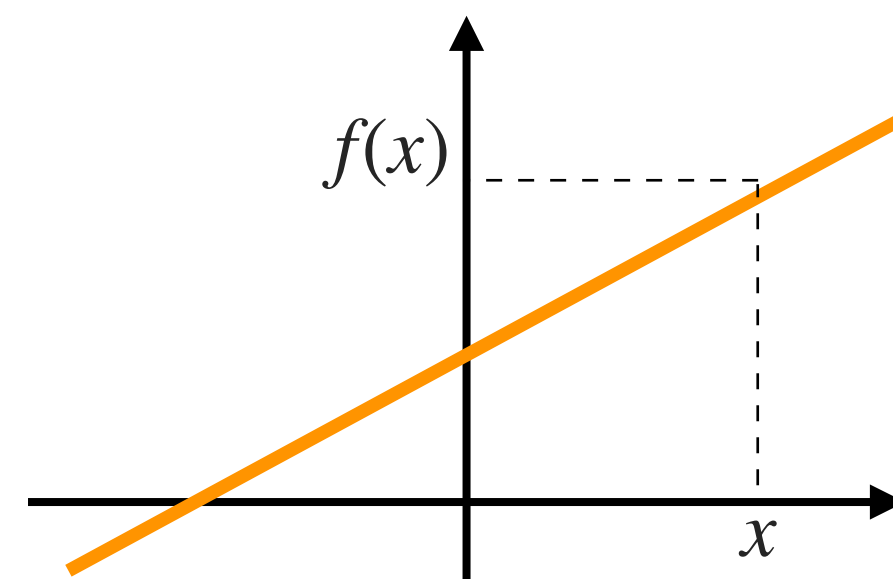
■ 対称性 (symmetry) : $\langle \mathbf{x}, \mathbf{y} \rangle = \langle \mathbf{y}, \mathbf{x} \rangle$

- 転置で表記すると少しだけ意外に見える : $\mathbf{w}^\top \mathbf{x} = \mathbf{x}^\top \mathbf{w}$

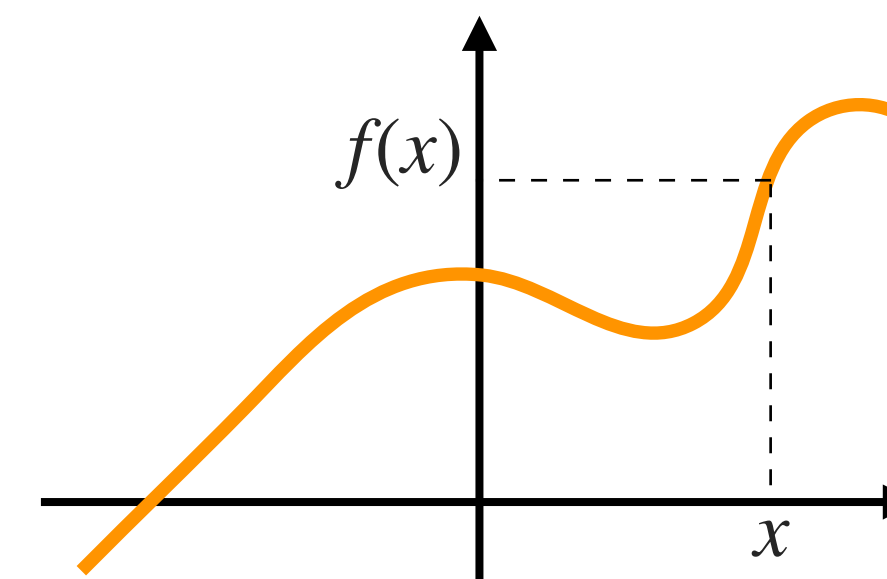
もっと複雑なモデル

もっと表現能力が高い複雑なモデルを使いたい

- 入力に対して非線型なモデルも使いたい（例：多項式回帰）



線型単回帰



非線型単回帰

- 画像／テキストなどを入力にする回帰モデルも作りたい



f_{θ}



7

キュウリの品質

“こんな映画には
滅多に出会えない”



f_{θ}



4.9

レーティング予測

線型モデル

特徴量 $\phi(x)$ とパラメーター θ の内積を使って表されるモデル

■ パラメーター線型モデル (linear-in-parameter model)

$$f_{\theta}(x) := \theta^{\top} \phi(x) \quad \left(= \sum_{b=0}^B \theta_b \phi_b(x) \right)$$

$$x \in \mathcal{X}, \phi_b(x) : \mathcal{X} \rightarrow \mathbb{R}$$

$$\phi(x) = \begin{pmatrix} \phi_0(x) \\ \vdots \\ \phi_B(x) \end{pmatrix} : \text{変換された特徴量}$$

- ここで, $\phi = (\phi_b)_{b=0}^B$ は任意の関数であり, 特徴写像 (feature map) と呼ぶ.
- $\theta \in \mathbb{R}^{B+1}$ は係数 (coefficient) や重み (weight) と呼ぶことがある.

【補足】ただし, $(\phi_b)_{b=0}^B$ は線型独立であることを仮定する. 仮に線型独立でなければ, 最初から従属なものを除外しておいても表現力は変わらない. 任意の関数と書いたが理論解析の際には多少の正則条件は課される. 【用語】 $(\phi_b)_{b=0}^B$ は基底関数 (basis functions) とも呼ばれるが, これは数学の「基底」とは違う意味なので注意.

線型モデル > 注意

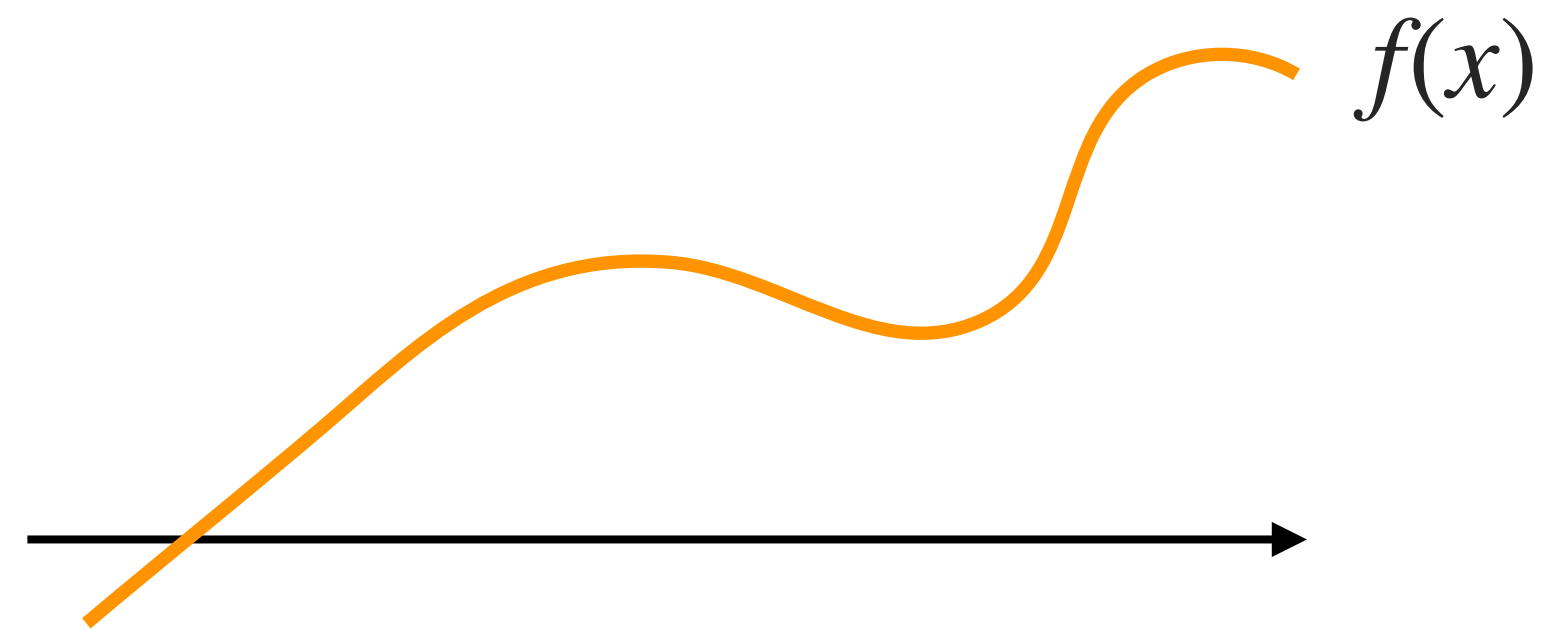
線型といってもパラメーター θ に関して線型であればよい

$$f_{\theta}(x) = \theta^{\top} \phi(x)$$

- 特徴量変換 ϕ は非線型でもよい，つまり一般に入力 x に関しては非線型.
 - 特に， x に対しても線型なモデル $f_{\theta}(x) = \theta^{\top} x$ は，データ線型モデル（linear-in-data model）と呼んで区別する
- この講義で単に線型モデルといったときは，パラメーター線型モデルを指す.

線型モデル

線型モデルの例としては、多項式モデルなどがある



- こんなに曲がっているのに線型とはどういうこと？

線型モデルの例 > 多項式モデル

入力に対しては非線型でも、パラメーターに対しては線型

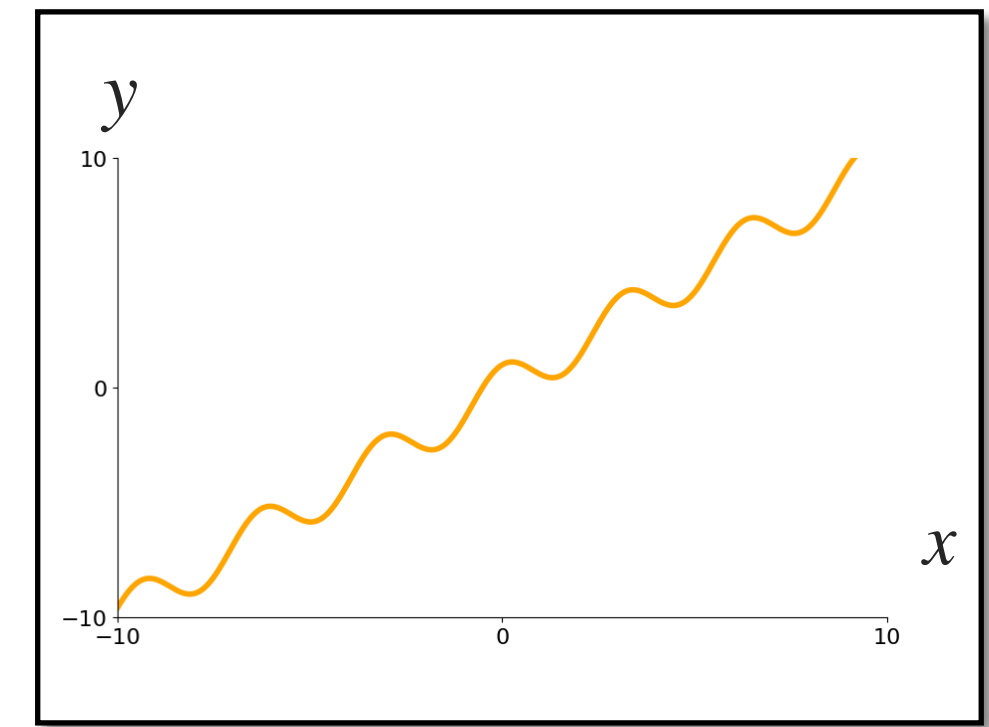
- r 次多項式モデル (r -th order polynomial model)

$$f_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \dots + \theta_r x^r$$

- 多項式の次数 r を上げれば複雑なモデルを表現できる

- これは、特徴写像を次のように定義すれば、 $f_{\theta}(x) = \boldsymbol{\theta}^{\top} \boldsymbol{\phi}(x)$ と表せる

$$\boldsymbol{\phi}(x) = \begin{bmatrix} 1 \\ x \\ \vdots \\ x^r \end{bmatrix}$$



【補足】ストーン・ワイエルシュトラスの定理 (Stone-Weierstrass theorem) により、多項式モデルで任意の連続関数を近似することができる (任意の有界区間で任意の精度まで、一様ノルムでの近似が、十分に大きい k を用いれば成立)

線型モデルの例＞三角多項式モデル

色々な特徴写像がある

- 三角多項式 (trigonometric polynomial) モデル

$$\phi(x) = \begin{bmatrix} 1 \\ \sin(x) \\ \cos(x) \\ \vdots \\ \sin(kx) \\ \cos(kx) \end{bmatrix}$$

- 一定間隔で特徴量が同じ値になるので、周期性をもったモデルになる

$$f_{\theta}(x) = \theta_0 + \theta_{1s} \sin(x) + \theta_{1c} \cos(x) + \cdots + \theta_{ks} \sin(kx) + \theta_{kc} \cos(kx)$$

【補足】 ストーン・ワイエルシュトラスの定理 (Stone-Weierstrass theorem) により、三角多項式モデルで任意の連続関数を近似することができる (任意の有界区間で任意の精度まで、一様ノルムでの近似が、十分に大きい k を用いれば成立) (Powell, 1981, Theorem 13.1).

線型モデルの例 > 多変数多項式モデル

色々な特徴写像がある

■ 多変数の多項式

$$\phi(x, y) := (1, \underbrace{x, y}_{\text{degree 1}}, \underbrace{x^2, xy, y^2}_{\text{degree 2}}, \dots, \underbrace{x^r, x^{r-1}y, \dots, xy^{r-1}, y^r}_{\text{degree } r})^\top$$

- あらかじめ交差項をこのように入れておけば, 多変数の多項式でも表せる

$$f_{\theta}(x, y) = \theta_0 + \theta_{1,0}x + \theta_{1,1}y + \theta_{2,0}x^2 + \theta_{2,1}xy + \theta_{2,2}y^2 + \dots + \theta_{r,0}x^r + \dots + \theta_{r,r}y^r$$

線型モデルの例＞Bag-of-words特徴量＋線型モデル

色々な特徴写像がある

- まず、語彙一覧 (vocabulary) を用意しておく

演奏, 演技, 演歌, 漢方, 漫画, ……

- 続いて、語彙に含まれる単語全ての出現回数をカウントして、特徴量として使う

入力

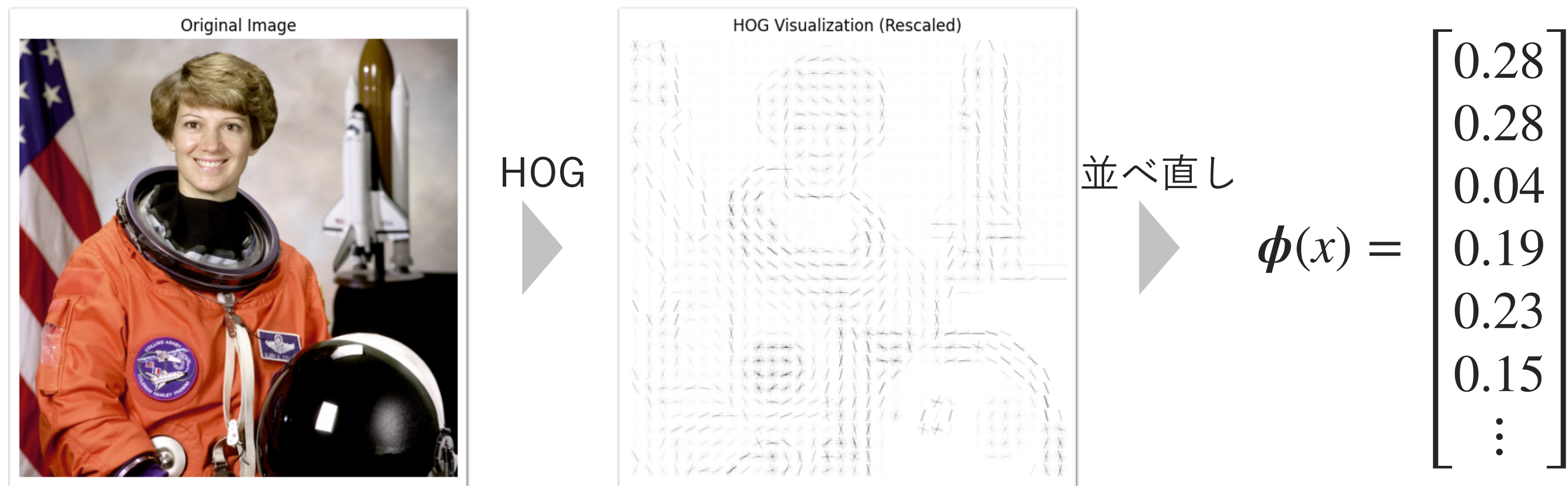
本日、老舗の演歌歌手が伝統と革新を融合させた舞台上、情感あふれる演技とともに演歌の名曲を披露し、観客から熱い拍手を浴びました。

→ {演奏: 0, 演技: 1, 演歌: 2, 漢方: 0, 漫画: 0, ……} なので, $\phi(x) = \begin{bmatrix} 0 \\ 1 \\ 2 \\ 0 \\ 0 \\ \vdots \end{bmatrix}$

線型モデルの例＞HOG特徴量＋線型モデル

色々な特徴写像がある

- HOG (Histogram of Oriented Gradients) 記述子
 - 画像の輝度勾配ヒストグラムを元にした特徴量（仕組みの説明は省略）



- 写真に人が写っているか否かの判定など

【実験コード】 [番号]_feature_map_HOG.ipynb 【画像出典】 <https://www.flickr.com/photos/nasacommons/16504233985/>

【参考】 Dalal, N., & Triggs, B. (2005). Histograms of oriented gradients for human detection. 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05), 1, 886–893 vol. 1. <https://doi.org/10.1109/CVPR.2005.177>

特徴量エンジニアリング



特徴量変換器を実際の応用に合わせて作ることは非常に重要

- 特徴量変換の作り込みを特徴量エンジニアリング (feature engineering) という
 - 応用ごとに作り込みを行うもの。性能にも寄与することがよくある。
 - 職人芸な面がある
- 一方、自前の特徴量変換器を作らなくてもよい場合が増えている
 - 事前学習済みの特徴量抽出器をダウンロードして使えば、基本的なタスクは解けることがある

予測タスクにおける線型モデルの恩恵



パラメーター線型モデルは扱いやすい道具として今でも使われることがある

■ 線型だと何が良いか？

- 理論的な取り扱いが簡単
- 最適化が簡単にできる場合も多い
- ϕ が（今のタスクを解くために）優れていれば、これだけで十分な表現力あり

■ 弱点

- モデルの性能限界がすべて ϕ に懸かっているので、 ϕ によっては高性能が出ない

線型モデルの最小二乗法

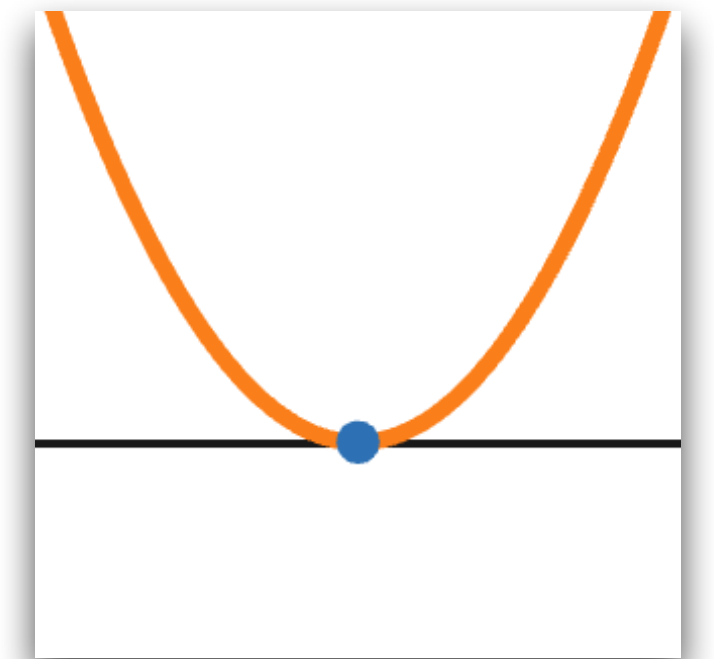
線型モデルでの最小二乗法の解

一般の線型モデルでの最小二乗法の解を求めてみよう

- 線型モデル $f_{\theta}(x) = \theta^{\top} \phi(x)$ の場合に、最適化問題

$$\text{Min}_{\theta \in \mathbb{R}^{B+1}} \frac{1}{2} \sum_{i=1}^n (y_i - f_{\theta}(x_i))^2$$

を解き、解となるパラメーター $\hat{\theta}$ を求めよ.



線型モデルでの最小二乗法の解

解の求め方の流れ

1) 目的関数は $L(\boldsymbol{\theta}) = \frac{1}{2} (\boldsymbol{\Phi}\boldsymbol{\theta} - \mathbf{y})^\top (\boldsymbol{\Phi}\boldsymbol{\theta} - \mathbf{y})$ と書き直せることを確認せよ。ここで,

$$\boldsymbol{\Phi} := \begin{pmatrix} \boldsymbol{\phi}(\mathbf{x}_1)^\top \\ \vdots \\ \boldsymbol{\phi}(\mathbf{x}_n)^\top \end{pmatrix} = \begin{pmatrix} \phi_0(\mathbf{x}_1) & \cdots & \phi_B(\mathbf{x}_1) \\ \vdots & \ddots & \vdots \\ \phi_0(\mathbf{x}_n) & \cdots & \phi_B(\mathbf{x}_n) \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}$$

2) $\text{Min}_{\boldsymbol{\theta} \in \mathbb{R}^{B+1}} L(\boldsymbol{\theta})$ を解け。ただし, $(\boldsymbol{\Phi}^\top \boldsymbol{\Phi})^{-1}$ が存在することを仮定する。

【用語】 この $\boldsymbol{\Phi}$ を計画行列 (design matrix) とよぶ。 ヒント: $\nabla_{\boldsymbol{\theta}}(\boldsymbol{\theta}^\top \mathbf{A} \boldsymbol{\theta}) = 2\mathbf{A}\boldsymbol{\theta}$, $\nabla_{\boldsymbol{\theta}}(\mathbf{b}^\top \boldsymbol{\theta}) = \mathbf{b}$ を使う。

【参考】 自力で解いてみる場合, Petersen, K. B., & Pedersen, M. S. (2012, November). The Matrix Cookbook. Technical University of Denmark. <http://www2.compute.dtu.dk/pubdb/pubs/3274-full.html> を片手に取り組むとよい。

線型モデルでの最小二乗法の解

必要条件である一階の条件を適用する

- **勾配 (gradient)** : 偏微分を並べた縦ベクトル (パラメーター数と同じ次元)

$$\nabla_{\theta} L(\theta) = \begin{pmatrix} \frac{\partial L(\theta)}{\partial \theta_0} \\ \vdots \\ \frac{\partial L(\theta)}{\partial \theta_B} \end{pmatrix}$$

- 最適解の必要条件として、一階の条件がある

- 制約なし最適化の場合 : $\nabla L(\theta) = \mathbf{0}$

今回はこちら

- 制約付き最適化の場合 : KKT条件 (Karush–Kuhn–Tucker conditions)

線型モデルでの最小二乗法の解

最適解が満たす方程式（一階の条件）：勾配が0となる点を探す

$$\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial L}{\partial \theta_0} \\ \vdots \\ \frac{\partial L}{\partial \theta_B} \end{pmatrix} = \boldsymbol{\Phi}^\top \boldsymbol{\Phi} \boldsymbol{\theta} - \boldsymbol{\Phi}^\top \mathbf{y} = \mathbf{0}$$

- これを解いて

$$\hat{\boldsymbol{\theta}} = (\boldsymbol{\Phi}^\top \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^\top \mathbf{y}$$

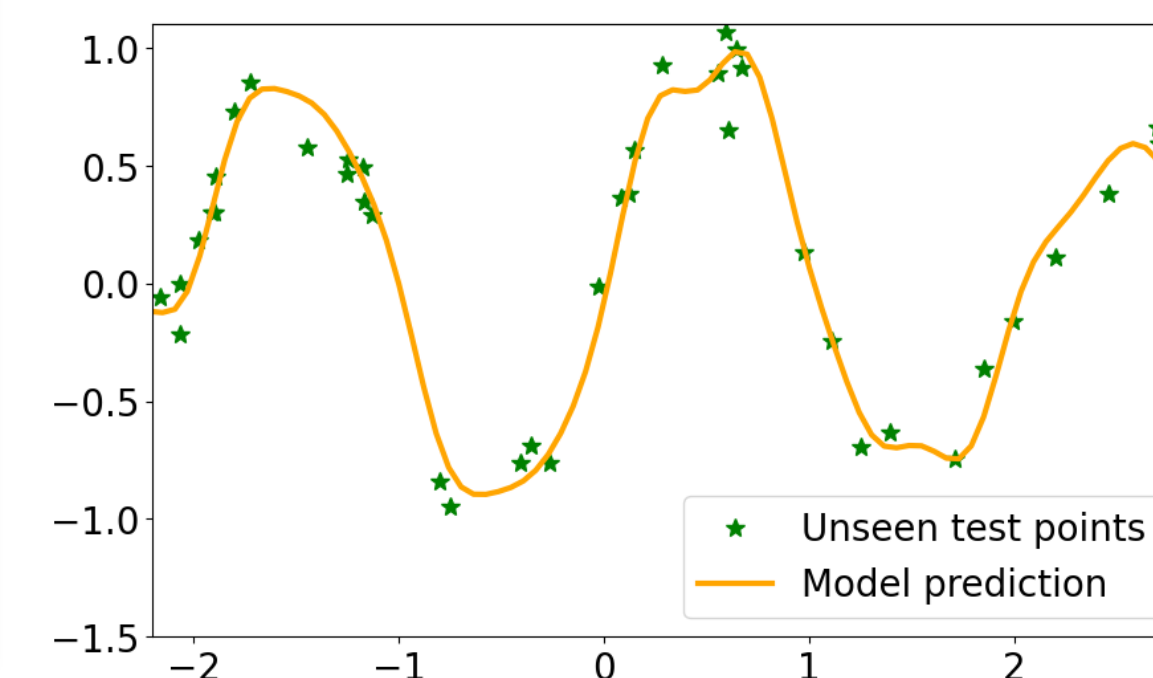
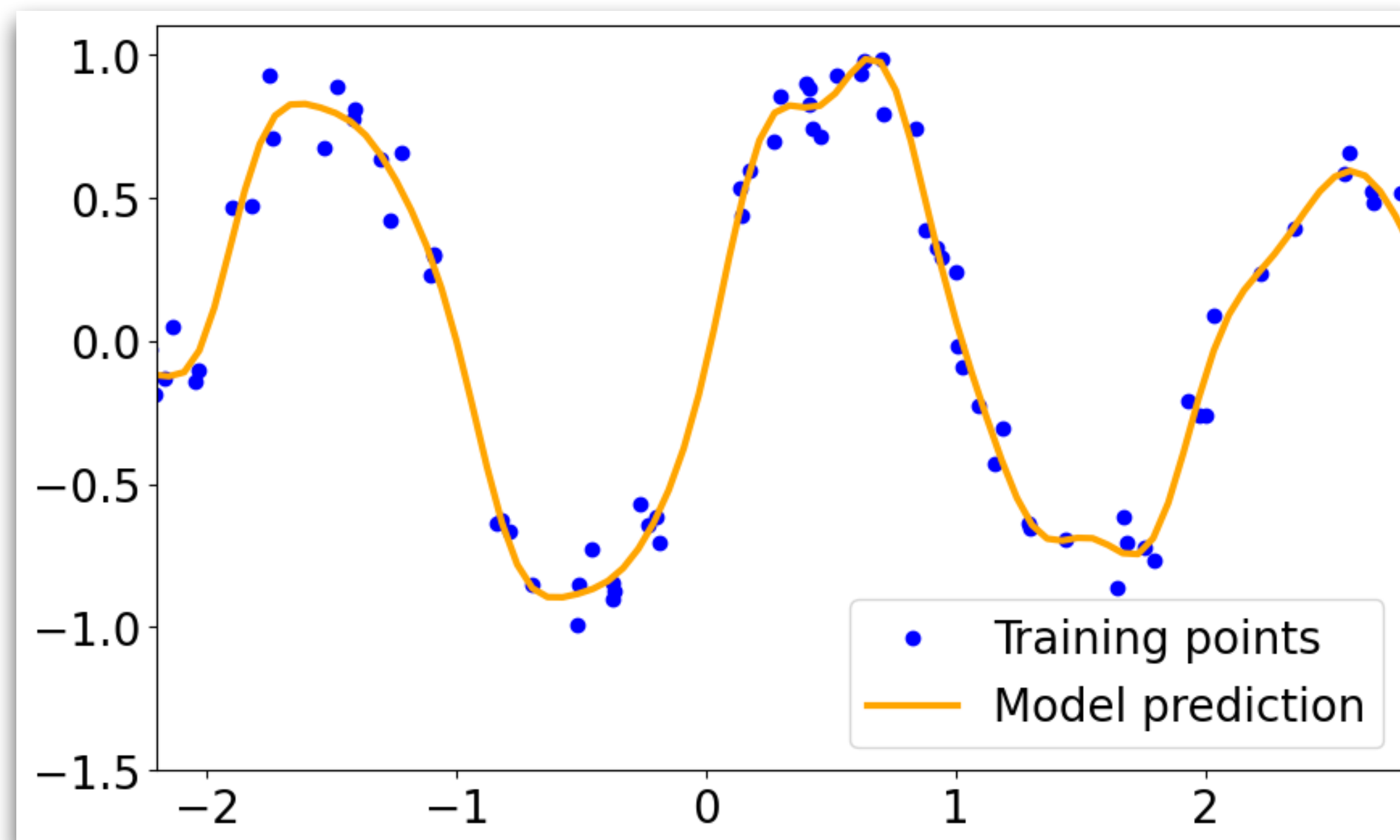
- 最適解が閉じた形式で求まった

【用語】 線型最小二乗回帰における $\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}) = \mathbf{0}$ という式を正規方程式（normal equation）という。正規方程式という用語の由来：方程式 $\boldsymbol{\Phi}^\top (\boldsymbol{\Phi} \boldsymbol{\theta} - \mathbf{y}) = \mathbf{0}$ は、 $(\boldsymbol{\Phi} \boldsymbol{\theta} - \mathbf{y})^\top \boldsymbol{\Phi} = \mathbf{0}^\top \Leftrightarrow \langle \boldsymbol{\Phi} \boldsymbol{\theta} - \mathbf{y}, \boldsymbol{\Phi}(\cdot) \rangle = 0$ ，すなわち $(\boldsymbol{\Phi} \boldsymbol{\theta} - \mathbf{y}) \perp \text{range}(\boldsymbol{\Phi})$ と同値。つまり、 $\text{range}(\boldsymbol{\Phi})$ に対する法線ベクトル（normal vector）になっていることを表す方程式なので、上記の名前になっている。閉じた形式（closed-form）は、四則演算や指数・対数、冪乗（べき乗）等の「よく知られた」関数で表される式を指す通称。

線型モデルでの最小二乗法 > 実行例

三角多項式モデル $f_{\theta}(x) = \theta^{\top} \phi(x)$ を最小二乗法で学習した結果

- ここで, $\phi(x) = (1, \sin(x), \cos(x), \sin(2x), \cos(2x), \dots, \sin(15x), \cos(15x))^{\top}$.



- 評価データによる予測性能評価も妥当

過適合 (過学習)

これまでのあらすじ

- 予測器を作りたい



- モデル f_{θ} を選び、損失関数 ℓ を決め、期待損失が小さいパラメーター θ を探す

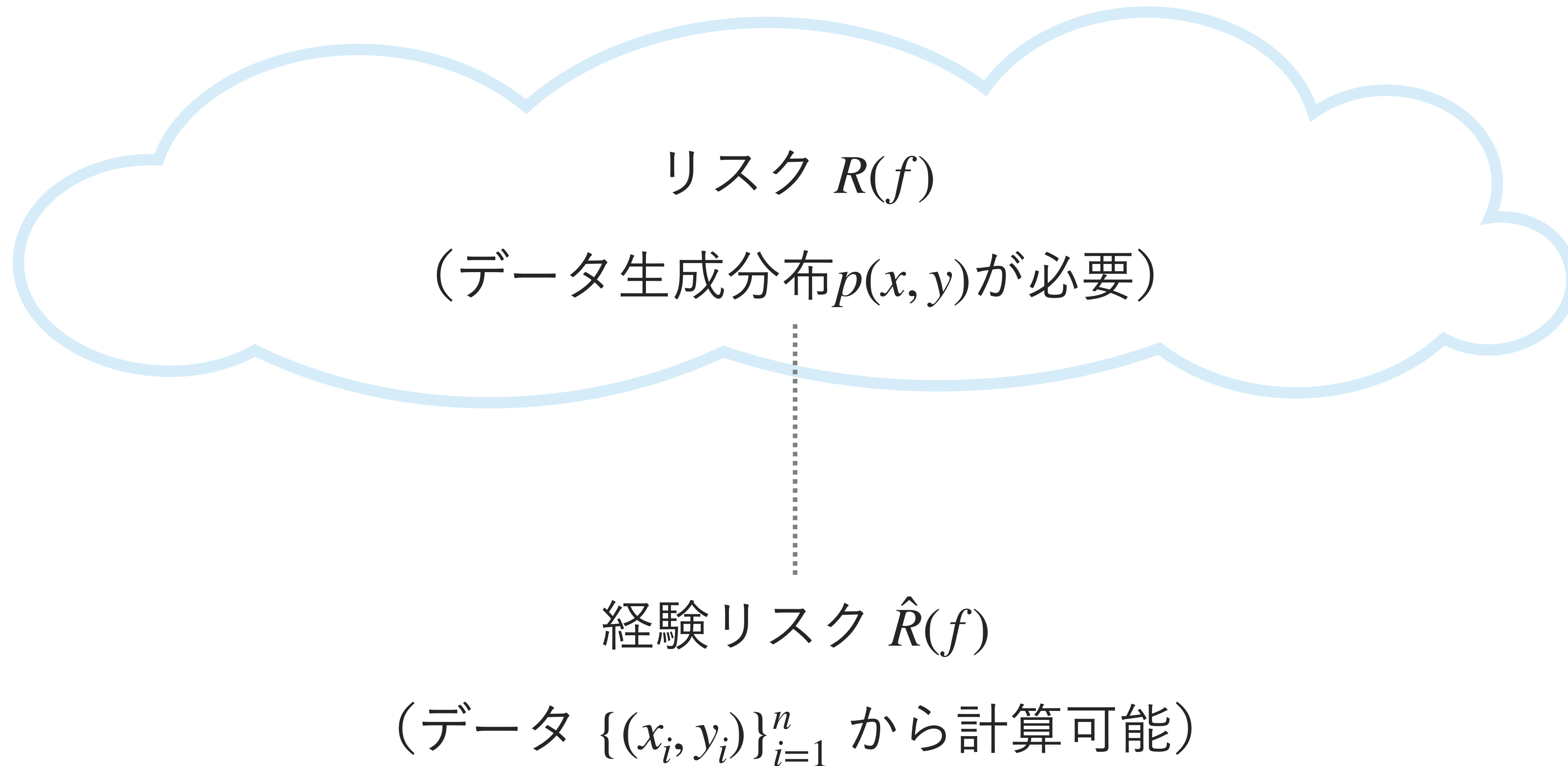
$$R(f_{\theta}) := \mathbb{E} [\ell(Y, f_{\theta}(X))] \quad \leftarrow \quad \text{計算不可能だが小さくしたい}$$

- 現実的には、損失のサンプル平均の最小化なら実行できることを紹介

$$L(\theta) := \hat{R}(f_{\theta}) := \frac{1}{n} \sum_{i=1}^n \ell(y_i, f_{\theta}(x_i)) \rightarrow \text{Minimize}$$

経験リスク

経験リスクは、リスク関数をサンプル近似したもの



経験リスク最小化 (Empirical Risk Minimization)

経験リスクを（何も手を加えずそのまま）目的関数として学習する

- 経験リスクそのものを目的関数として設定して最小化

$$L(\theta) = \hat{R}(f_{\theta}) \rightarrow \text{Minimize } \theta \in \Theta$$

- リスクのサンプル近似を最小化しているので、妥当そうではある
- しかし、愚直にこれだけを実行すると、過適合することがある

経験リスク最小化の限界



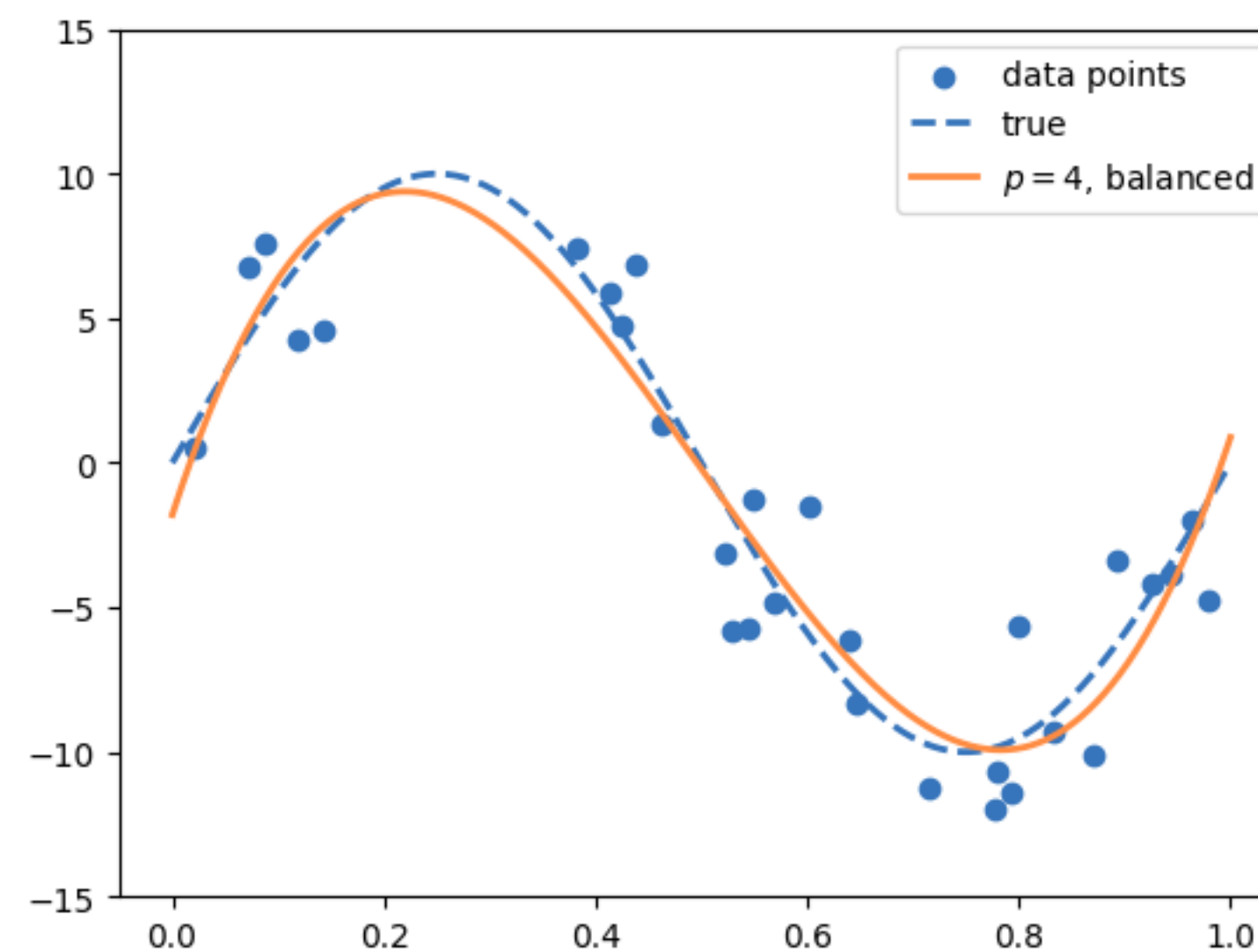
特に複雑なモデルでは、過適合・過学習という問題が起きるようになる

- 経験リスク最小化は、 $R(f_\theta)$ が小さくなることを願って、 $\hat{R}(f_\theta)$ を最小化する手法
 - しかし、狙い通りに $R(f_\theta)$ が小さくなるかどうかは別問題
 - 上手くいくときはうまくいくが、思惑通りにはいかないことの方が多い
- 特に、過適合・過学習という問題が生じる場合があることが知られている

経験リスク最小化

多項式モデルの最小二乗法での学習の成功例

- 4次関数でうまく予測できるような生成分布のデータで実験
 - 4次関数モデルを最小二乗法で学習した結果, よいモデルができた

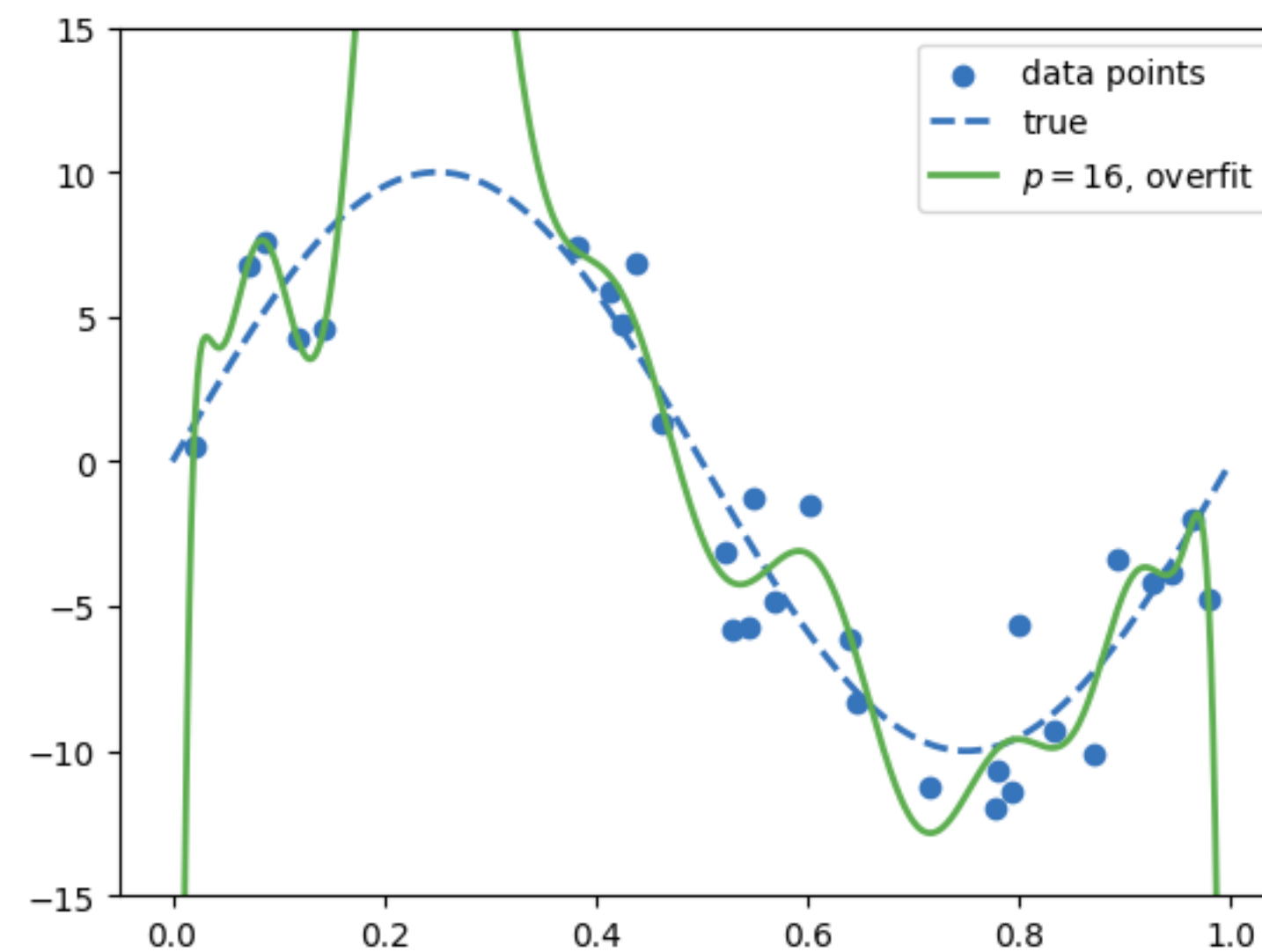


4次関数モデル

過適合（過学習）

パラメーターが多いモデルを単なるERMで学習させると、**過適合**が生じがち

- **過適合（overfit）**：経験リスクは小さくなったが、リスクが小さくない状態
 - つまり、見かけ上の学習はしているが、汎化しないモデルになっている状態



16次関数モデル

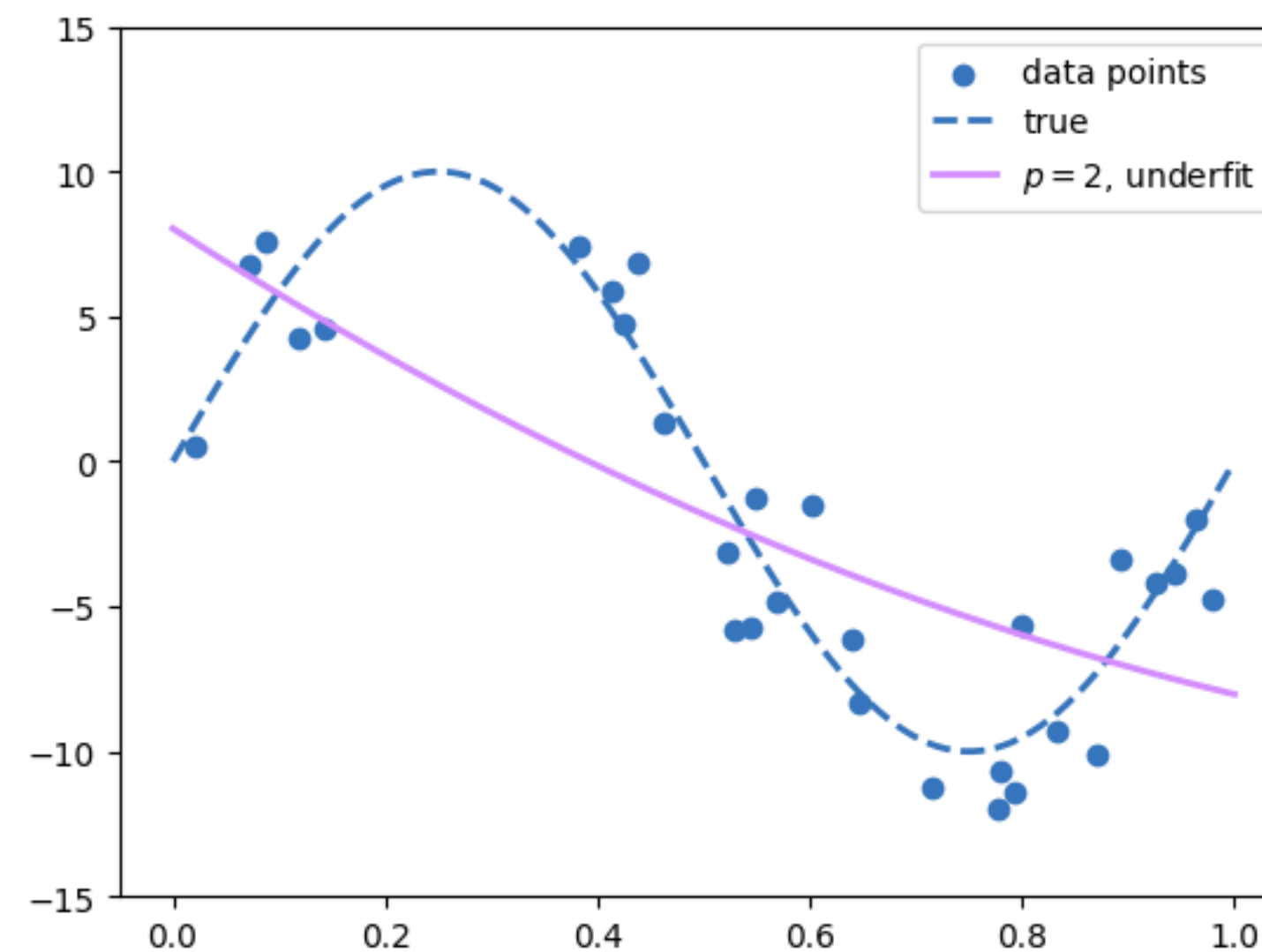
【用語】 過適合という訳のほうが英語とうまく対応するが、日本語では過学習という用語も浸透している。外来語としてオーバーフィットということもある。

【実験ソースコード】 [番号]_overfit_linear_model.ipynb

過少適合

パラメーターが少なすぎるモデルだけでやりくりするのも難しい

- **過少適合 (underfit)** : 最適化後でも経験リスクが大きいままの状態
 - つまり, モデルの柔軟さが足りず, そもそも学習が十分に進まない状態



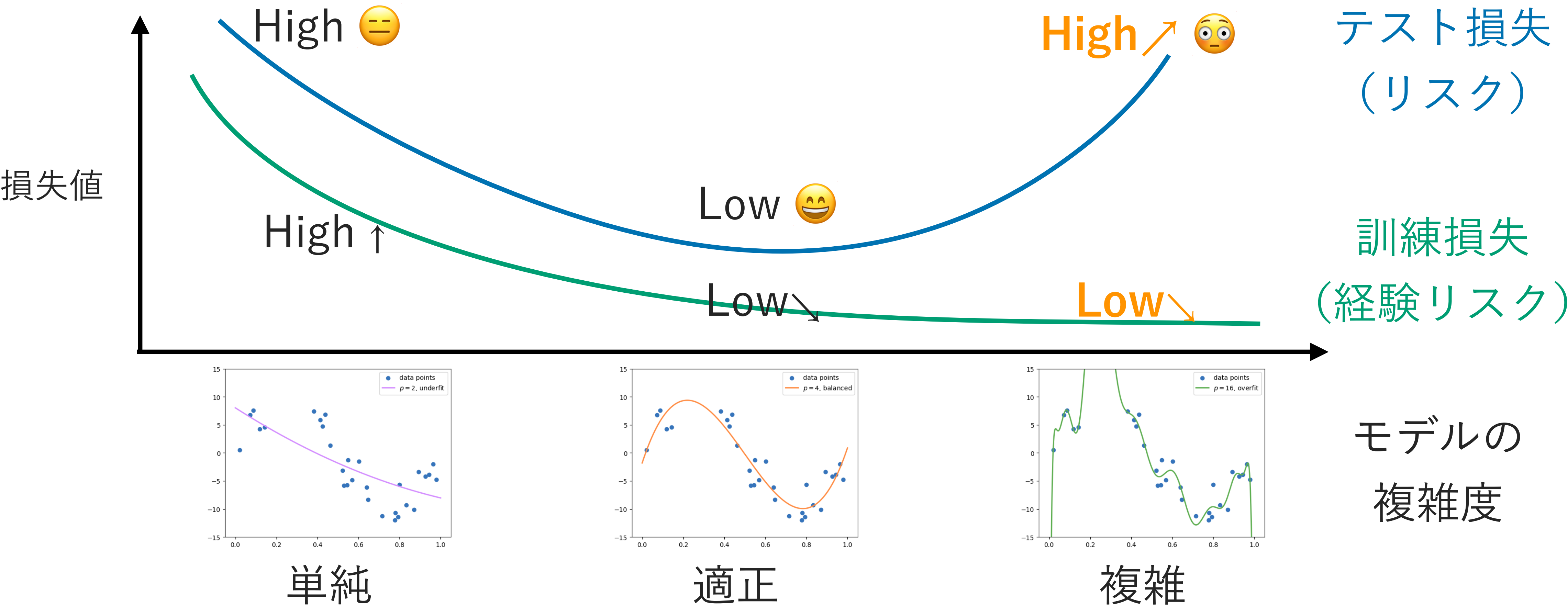
2次関数モデル

【用語】 外来語としてアンダーフィットということも多い.

【実験ソースコード】 [番号]_overfit_linear_model.ipynb

過適合現象（Overfitting）＞ 模式図によるまとめ

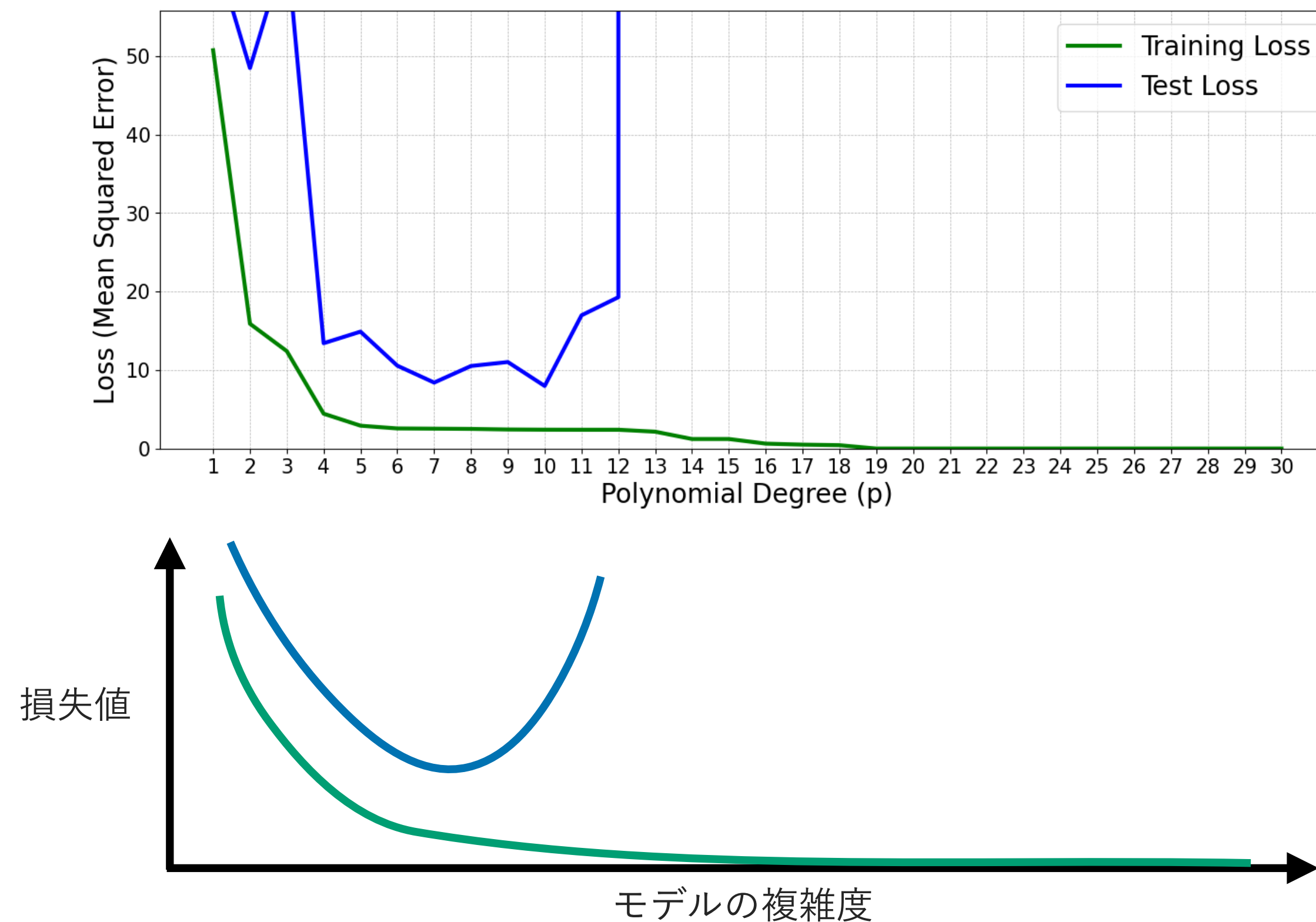
モデルの複雑度を上げていくと，リスクの値が再び上昇する現象



過適合現象（Overfitting）＞学習曲線（Learning Curve）

実験：多項式モデルの最小二乗回帰で次数を上げていった結果

- 横軸：多項式の最大次数，縦軸：損失値として，訓練・評価の損失を描いた図



【実験ソースコード】[番号]_learning_curve_overfitting.ipynb

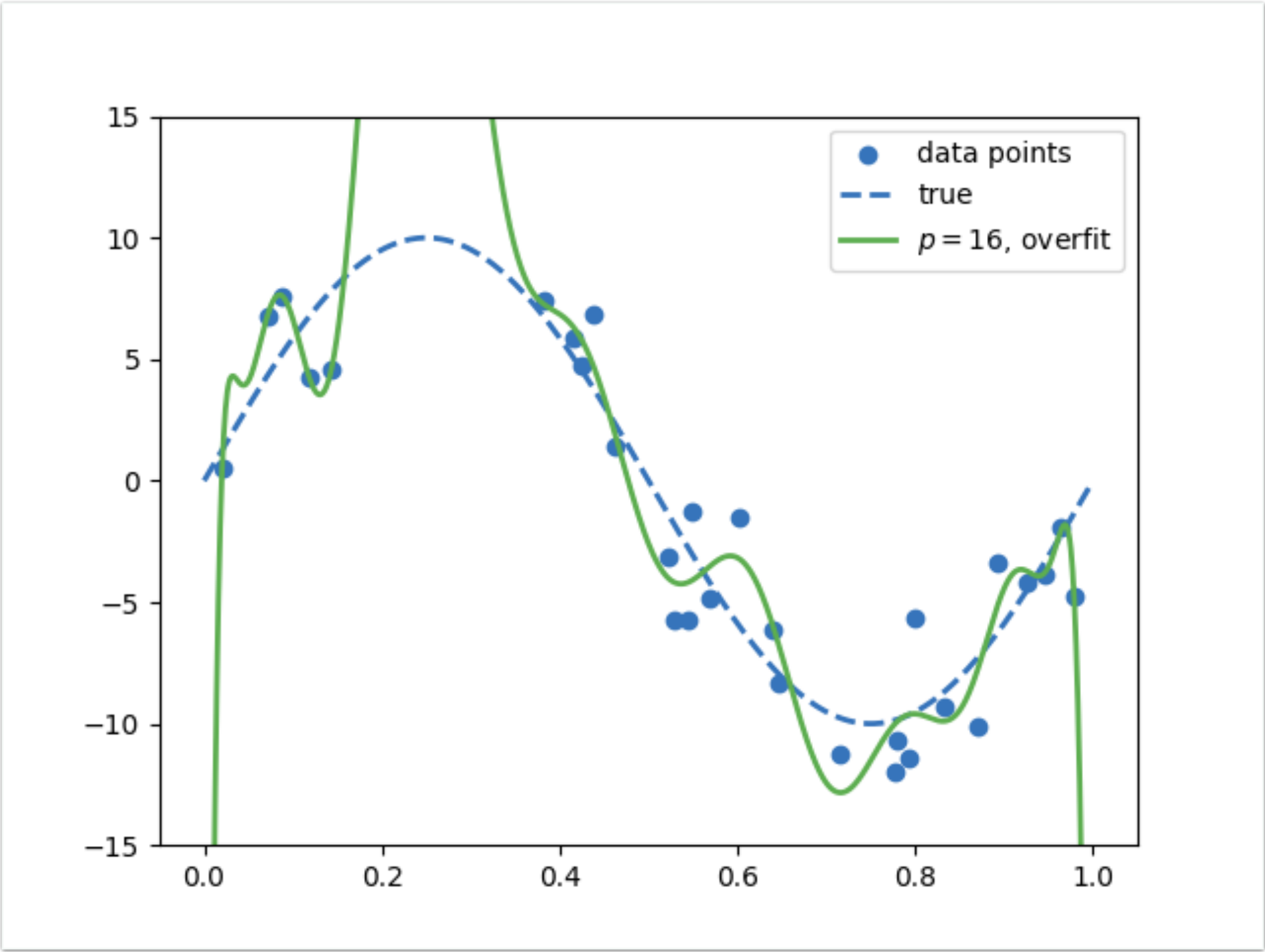
【用語】横軸が何らかのモデル複雑度，縦軸が何らかの指標値として，訓練と評価の結果をそれぞれプロットした曲線のことを，学習曲線という。

モデルの複雑度と正則化

多項式モデルの過適合とパラメーターの大きさ

上下に激しく変動する関数はパラメーターの絶対値が大きくなって発生

- 入力の微小な差に大きく反応 = パラメーターの絶対値が大きい
- 訓練データ点を通過するように辻褄を合わせていると考えられる



	r=2	r=4	r=16
θ_0	8.0	-1.8	-87.1
θ_1	-23.4	112.3	10466.1
θ_2	7.3	-327.4	-487005.0
θ_3		219.0	12103103.6
θ_4		-1.2	-180067412.9
θ_5			1723030777.0
θ_6			-11159264579.3
θ_7			50782142907.2
θ_8			-166670522119.7
θ_9			400990179401.2
θ_{10}			-712278442253.3
θ_{11}			932198468910.3
θ_{12}			-887330785442.5
θ_{13}			597242347925.3
θ_{14}			-269343089344.4
θ_{15}			73003666266.9
θ_{16}			-8989291594.0

基本知識の確認＞ベクトルの大きさ（ノルム）

ベクトルの大きさ（長さ）はノルムで測る

■ ノルム (norm) $\|x\|$

- 性質： $\|x\| \geq 0$, かつ, $(\|x\| = 0) \Leftrightarrow (x = \mathbf{0})$
- 性質： $\|\alpha x\| = |\alpha| \cdot \|x\|$ （斉次性）, $\|x + y\| \leq \|x\| + \|y\|$ （三角不等式）

■ 内積から誘導されるノルム $\|x\|_{\ell^2}$ （2-ノルム）

$$\|x\|_{\ell^2} = \sqrt{\langle x, x \rangle} = \sqrt{x_1^2 + x_2^2 + \cdots + x_d^2}$$

- 最も有名なノルム（断りがなければこれ）

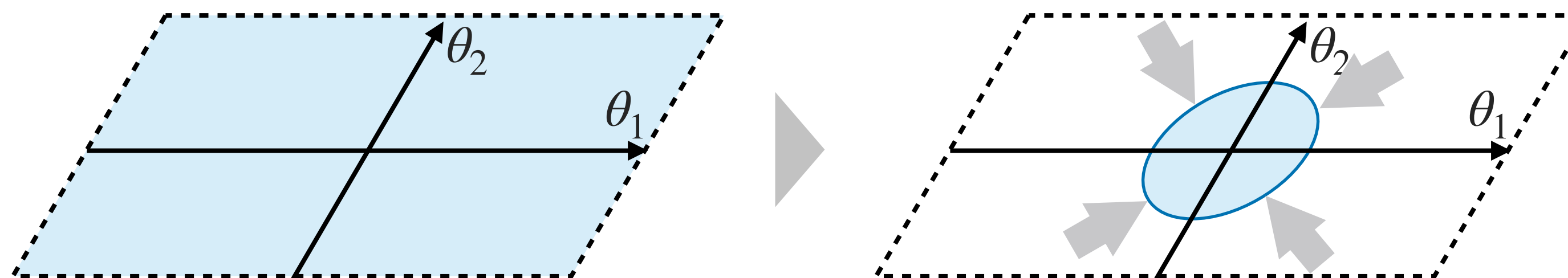
正則化＞L2正則化付き経験リスク最小化

2-ノルムを用いた罰則項を目的関数に追加して，過適合を防止

■ L2正則化（L2 regularization）

$$L(\boldsymbol{\theta}) := \underbrace{\hat{R}(f_{\boldsymbol{\theta}})}_{\text{損失項}} + \underbrace{\lambda \cdot \|\boldsymbol{\theta}\|^2}_{\text{正則化項}} \rightarrow \text{最小化}$$

- $\lambda \geq 0$ は**正則化係数（regularization coefficient）**という（学習時には固定）

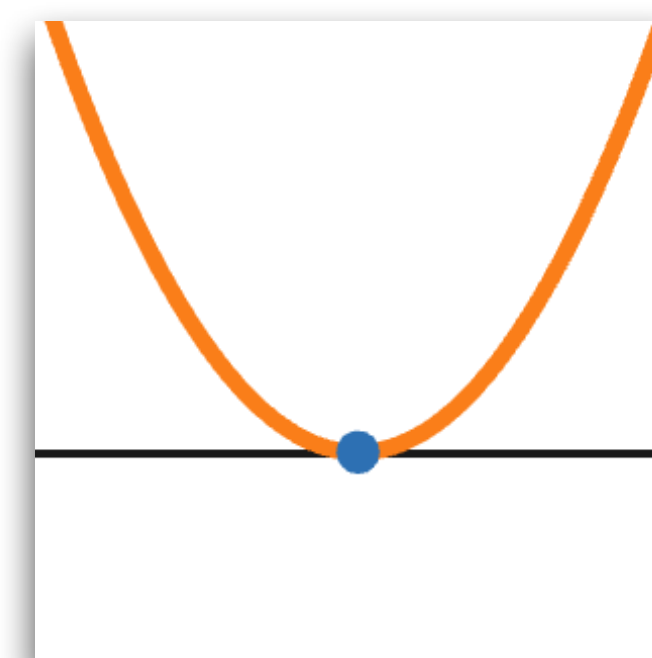


線型モデルでのL2正則化付き最小二乗法の解

一般の線型モデルでのL2正則化付き最小二乗法の解を求めてみよう

- 線型モデル $f_{\theta}(x) = \theta^{\top} \phi(x)$ の場合に、最適化問題

$$\text{Min}_{\theta \in \mathbb{R}^{B+1}} \frac{1}{2} \sum_{i=1}^n (y_i - f_{\theta}(x_i))^2 + \frac{\lambda}{2} \|\theta\|^2$$



を解き、解となるパラメーター $\hat{\theta}$ を求めよ。

- 記号として以下を使う

$$\Phi := \begin{pmatrix} \phi(x_1)^{\top} \\ \vdots \\ \phi(x_n)^{\top} \end{pmatrix} = \begin{pmatrix} \phi_0(x_1) & \cdots & \phi_B(x_1) \\ \vdots & \ddots & \vdots \\ \phi_0(x_n) & \cdots & \phi_B(x_n) \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}$$

正則化＞線型モデルのL2正則化付き最小二乗法

正則化無しの場合と同じ方法で最適解の公式を得られる

- 正則化付きの目的関数は、 $\lambda\|\boldsymbol{\theta}\|^2 = \lambda\boldsymbol{\theta}^\top\boldsymbol{\theta} = \boldsymbol{\theta}^\top(\lambda\boldsymbol{I})\boldsymbol{\theta}$ に注意すると、

$$L(\boldsymbol{\theta}) = \frac{1}{2}(\mathbf{y} - \boldsymbol{\Phi}\boldsymbol{\theta})^\top(\mathbf{y} - \boldsymbol{\Phi}\boldsymbol{\theta}) + \boldsymbol{\theta}^\top\left(\frac{\lambda}{2}\boldsymbol{I}\right)\boldsymbol{\theta}$$

- 一階の条件

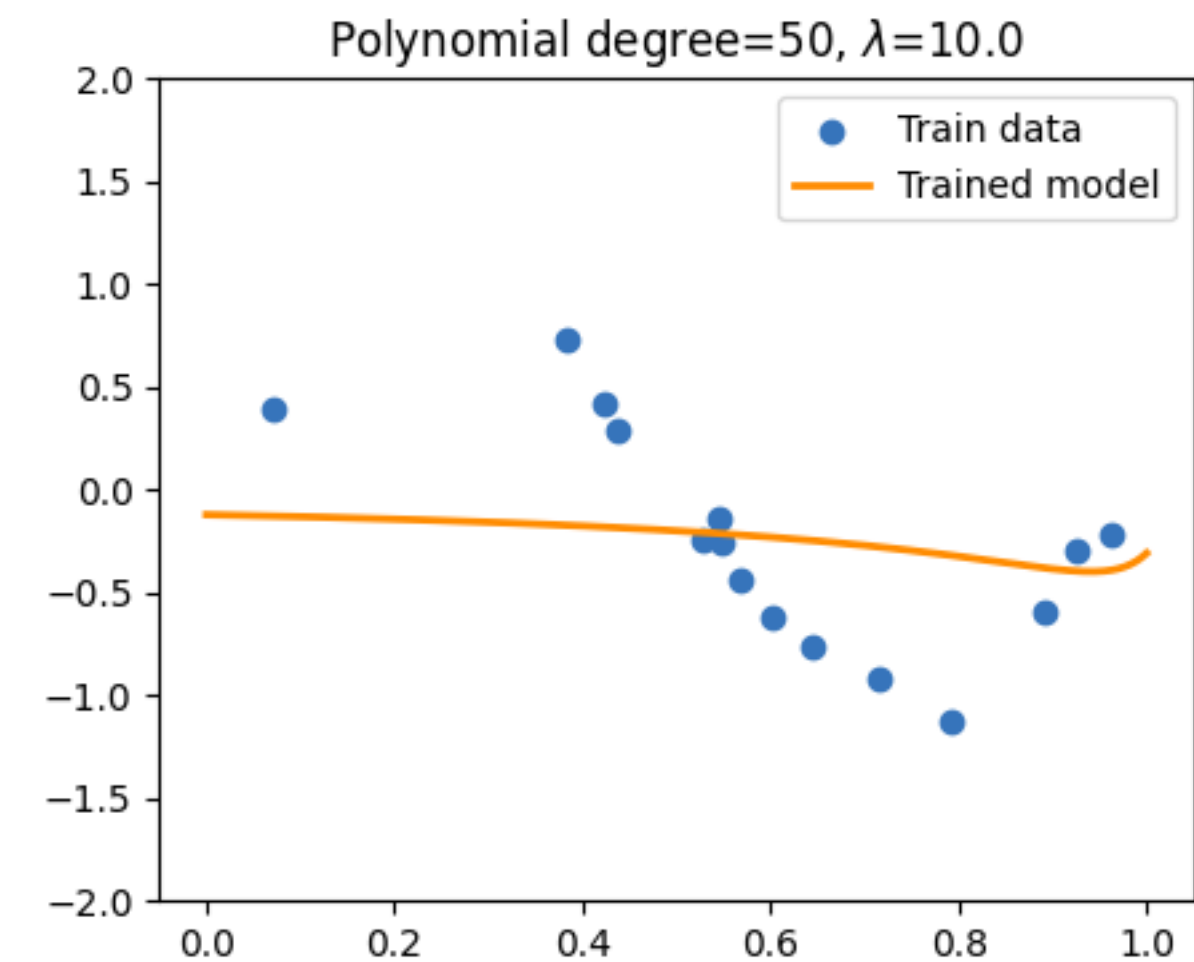
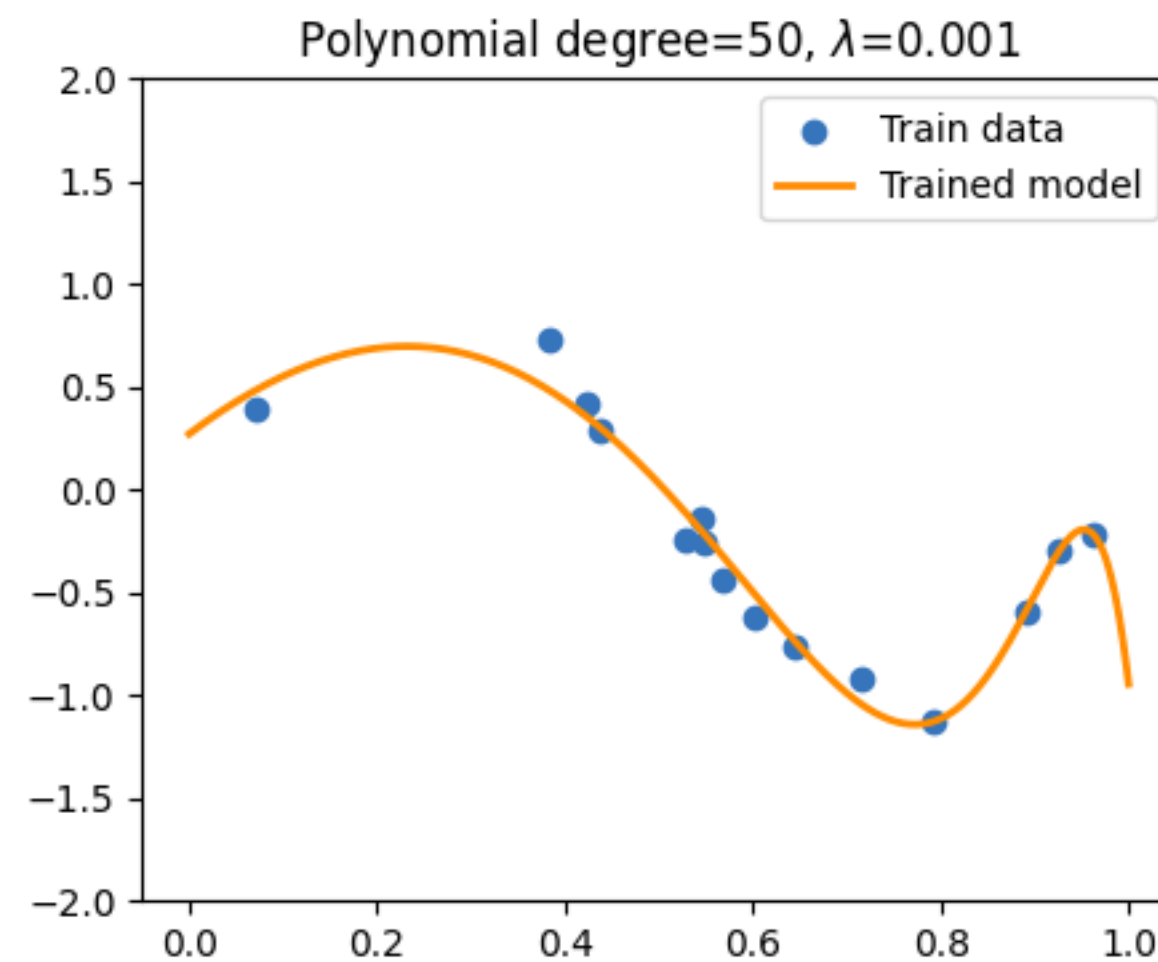
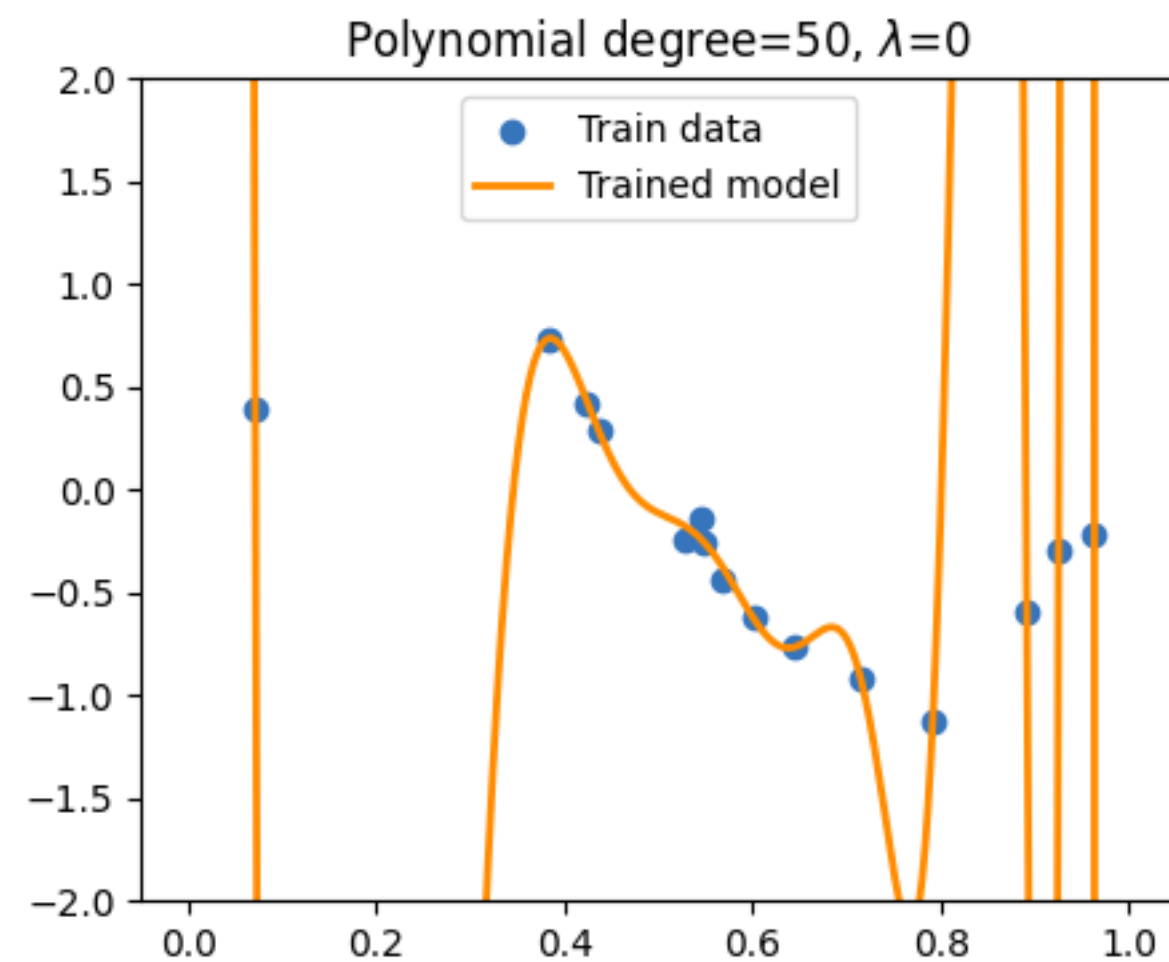
$$\nabla_{\boldsymbol{\theta}}L(\boldsymbol{\theta}) = (\boldsymbol{\Phi}^\top\boldsymbol{\Phi} + \lambda\boldsymbol{I})\boldsymbol{\theta} - \boldsymbol{\Phi}\mathbf{y} = \mathbf{0}$$

- よって、最適解の公式として次を得られる。

$$\hat{\boldsymbol{\theta}} = (\boldsymbol{\Phi}^\top\boldsymbol{\Phi} + \lambda\boldsymbol{I})^{-1}\boldsymbol{\Phi}^\top\mathbf{y}$$

正則化＞線型モデルのL2正則化付き最小二乗法＞実行例

L2正則化の強さを操作することで、過適合をコントロールできる



正則化無し 0 $\xrightarrow{\lambda}$ 正則化強

- 正則化係数 λ の大きさによって、正則化の強さを調整し、過適合を抑制できる

【補足】 正則化が強すぎると $\lambda \rightarrow +\infty$ で $\theta \simeq 0$ が解となり、例えば線型モデルなら定数関数になってしまう。

【実験ソースコード】 [番号]_l2_regularized_linear_regression.ipynb

正則化 (Regularization)

より単純なモデルが学習されるようにする仕掛けのこと

- 意図して組み込む明示的な正則化と，意図せず機能する暗黙的な正則化がある

明示的な正則化 (explicit regularization)

- 最適化問題の目的関数や制約式に手を加えるなど
- 例) L2正則化

暗黙的な正則化 (implicit regularization)

- 選んだ最適化法の傾向など明示的でない全ての正則化
- 例) 確率的勾配法の性質

過適合の防止



過適合を防止するには、モデルを適切に制限する必要がある

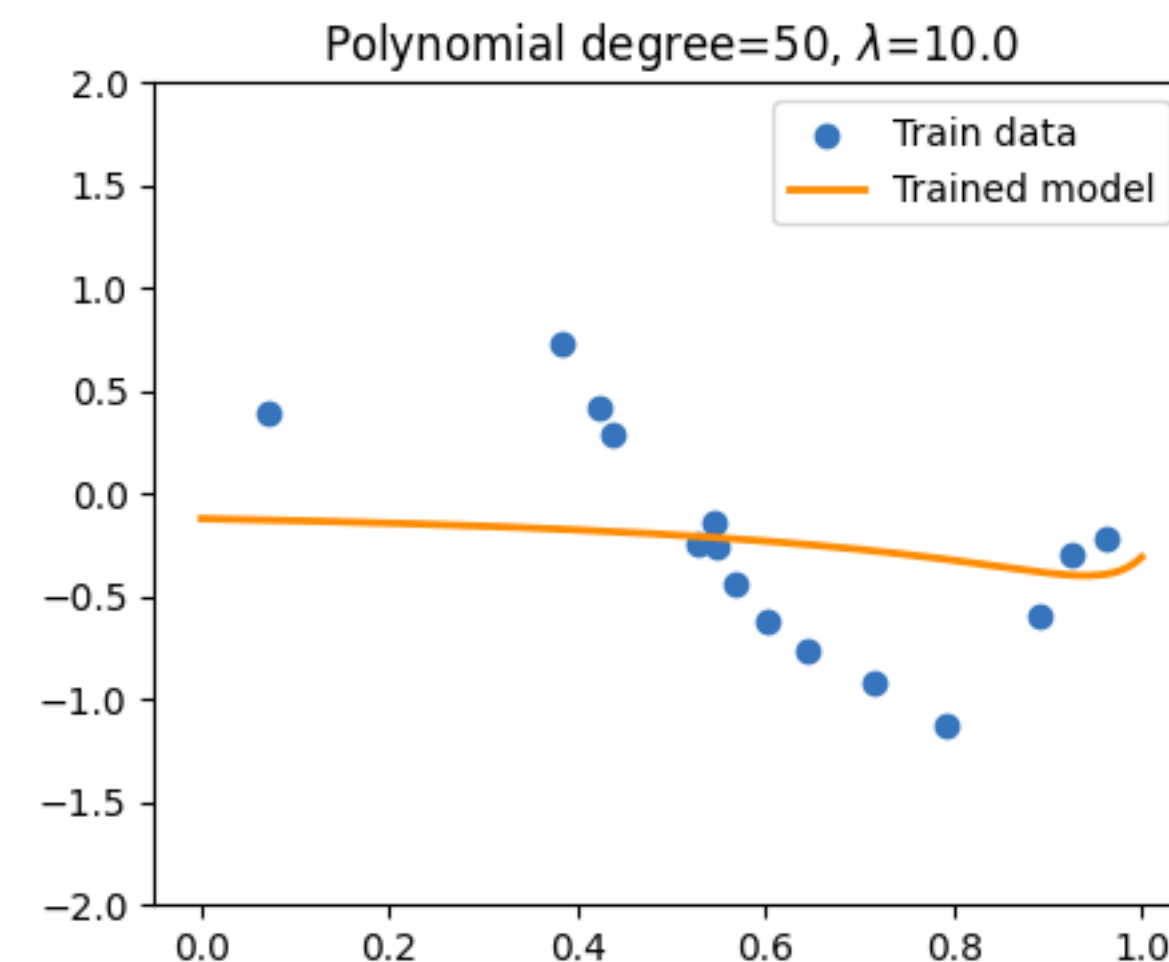
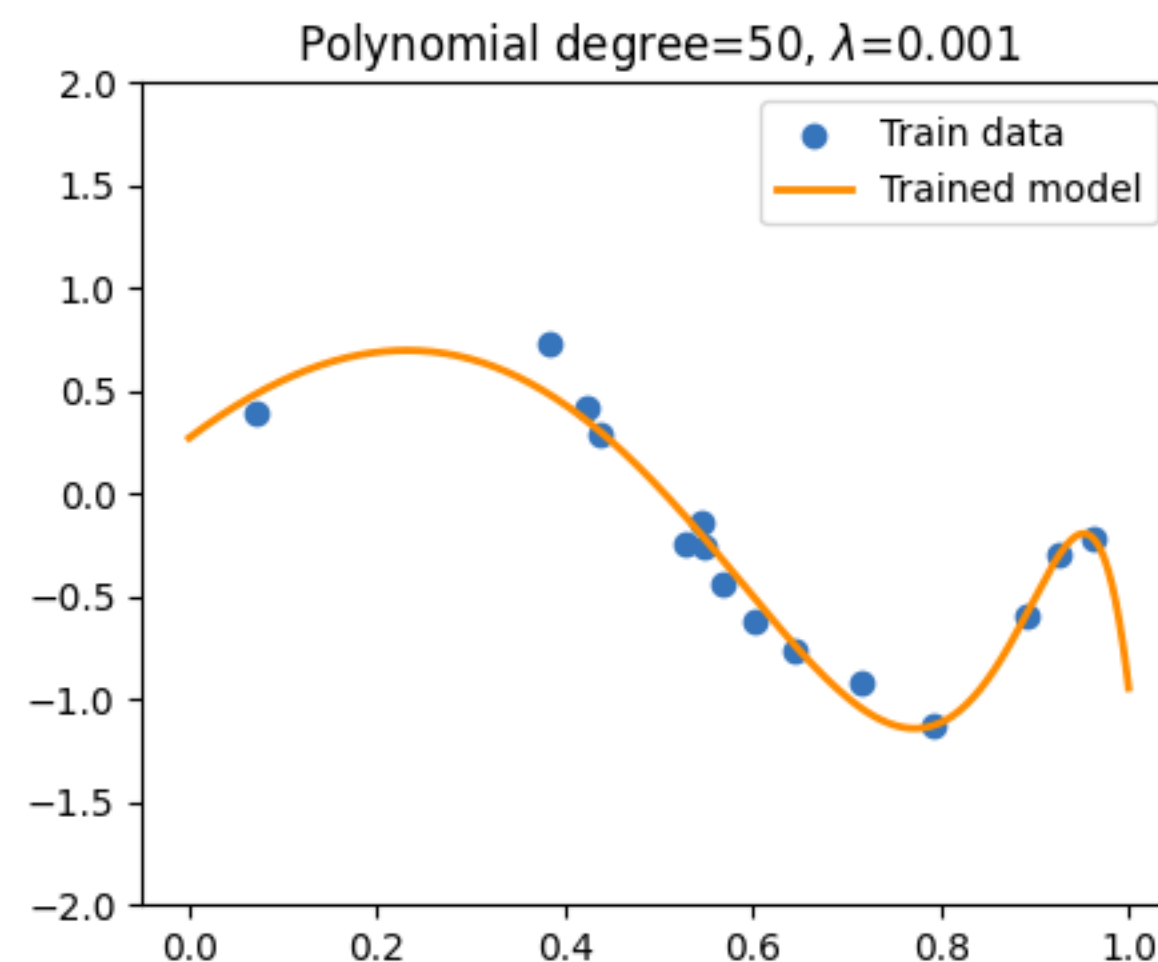
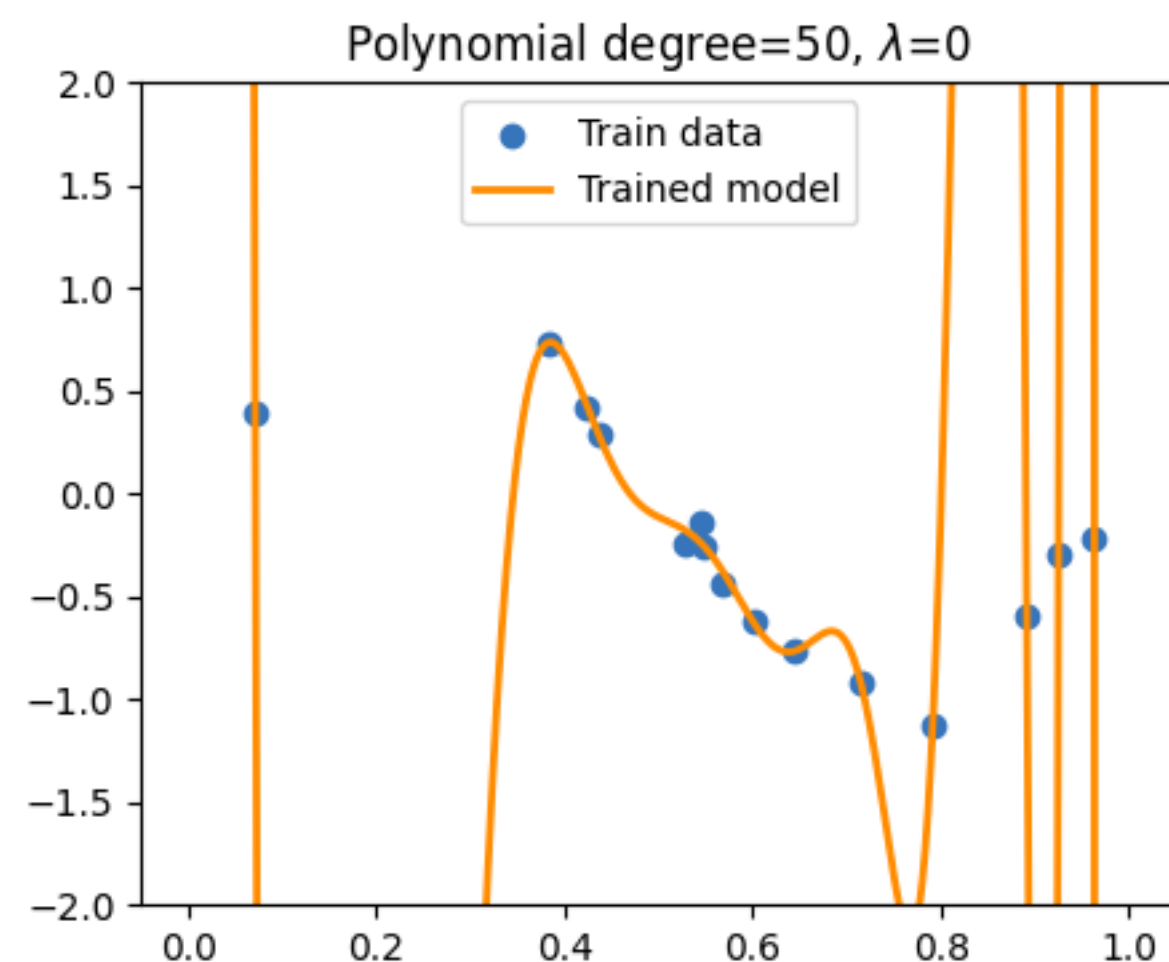
- モデルの複雑度を抑えるためには、いくつかの方法がある
 - モデルや学習過程の**正則化**を行う
 - モデルに固有の、複雑度を調整できる方法もある（例：多項式回帰の次数）
- どのぐらいの正則化を施せばよいか

モデル選択（ハイパーパラメーター選択）

正則化係数の選び方

L2正則化を使うとき、 λ の値はどのように決めればよいのか？

- 正則化係数 λ の大きさによって、正則化の強さを調整し、過適合を抑制できる



正則化無し 0 $\longrightarrow$ λ 正則化強

用語＞ハイパーパラメーター (Hyperparameters)

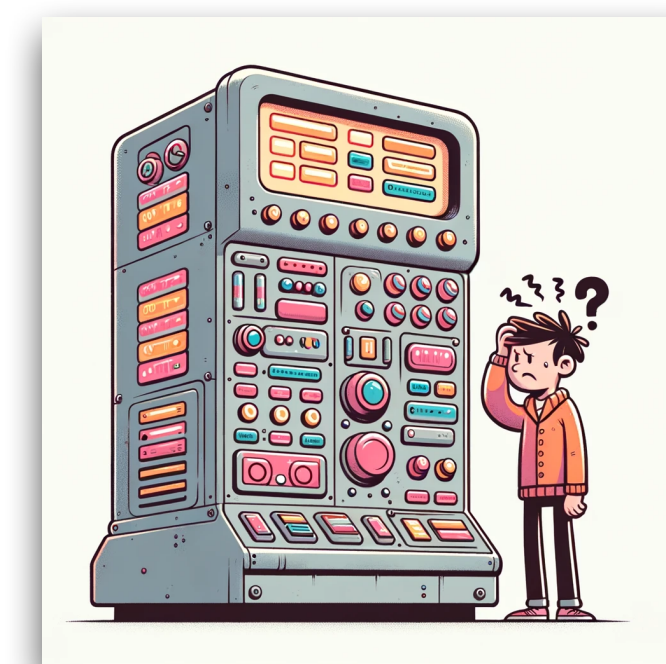
正則化係数の他にも、決めなくてはならないものは色々ある

■ ハイパーパラメーター (hyperparameters)

- モデルの訓練前に選んで固定するパラメーターのこと

■ 例

- モデルの複雑さに関わるもの（多項式モデルの最大次数 r など）
- 正則化に関わるもの（L2正則化の正則化係数 λ など）
- 最適化手法に関わるもの（勾配法の学習率 η など）
- そもそもどのようなモデルを使うか（モデル選択）
- そもそも特徴量として何を含めて何を含めないか（変数選択）



モデル選択（ハイパーパラメーター選択）

ハイパーパラメーター選択は、何らかのモデル選択規準を用いて行う

■ モデル選択規準（model selection criteria）

- モデルが複数あるとき、その中で最良のものを選択するための数値的指標
- 例）取り置き法（holdout method）、交差検証（cross-validation）、エビデンス（evidence）、情報量規準（information criteria）など

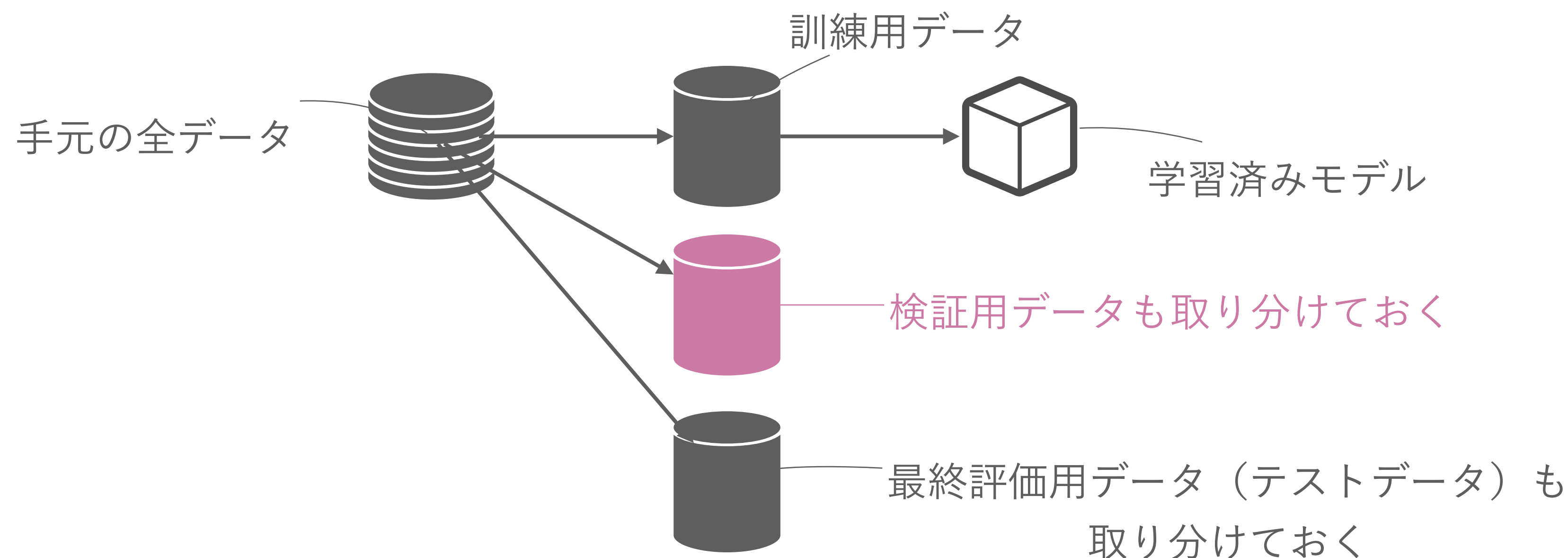
■ 共通する考え方（予測タスクの場合）

- 未知データで上手く予測できるモデルを作りたい、という基本に立ち返る

モデル選択規準＞検証データの取り置き法

基本の考え方

- まずはモデルの訓練時に使わない**検証データ（validation data）**を取り分ける

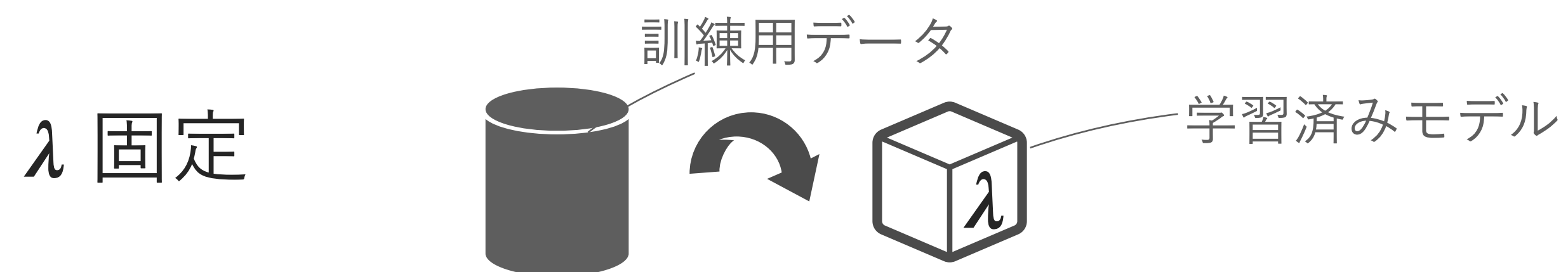


【用語】 検証用データのことを、開発用データ（development data）と呼ぶこともある。

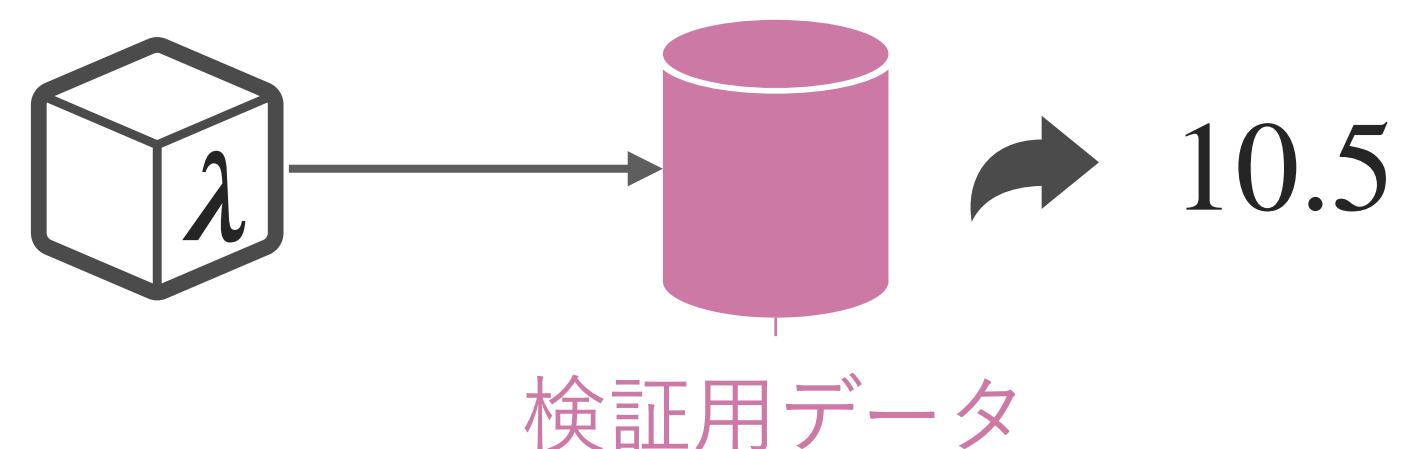
モデル選択規準＞検証データの置き置き法

ハイパーパラメーターを固定 → モデルを学習 → 検証用データで評価

- ハイパーパラメーターを固定し，モデルを学習



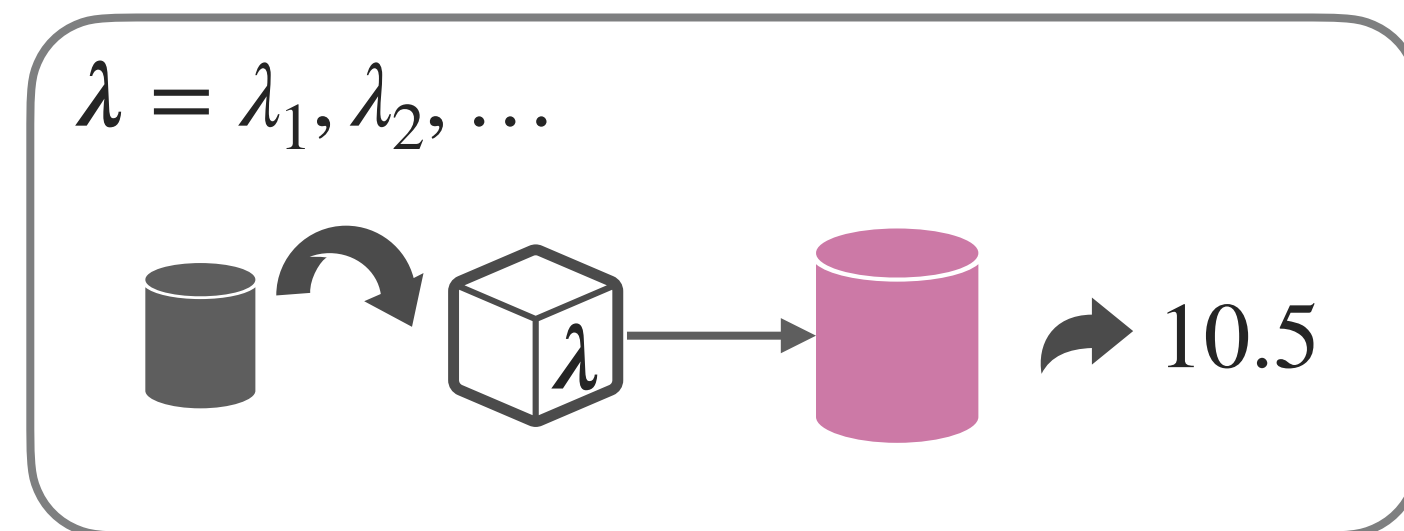
- 取り分けておいた検証用データで，性能を評価



モデル選択規準＞検証データの置き置き法

収集した検証評価結果をもとに，ハイパーパラメーターを選択する

- λ を変えながら，これを繰り返して，検証データでの評価結果を記録していく



記録
▶

λ	検証評価値
λ_1	10.5
λ_2	3.8
$\vdots$	$\vdots$

- 検証評価値が最小（最大）だった λ を，最適な λ だと考える
- こうして選んだ λ でのモデルを使う 

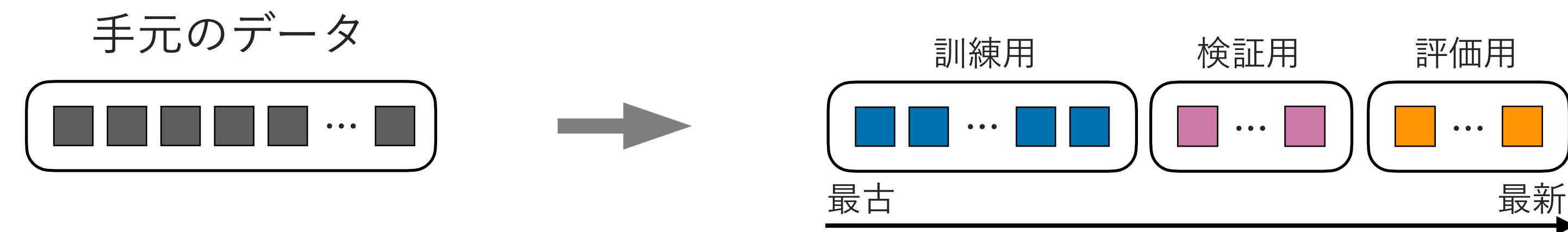
モデル選択規準＞置き置き法 (holdout method)

検証データの置き置きは、実際の性能のなるべく良い近似となるように行う

- 無作為分割 (random split) (例) 訓練用:検証用:評価用=8:1:1で置き置き



- 時系列分割 (time-series split) (例) 最新1ヶ月のデータを評価用に、その前の1ヶ月のデータを検証用に置き置き



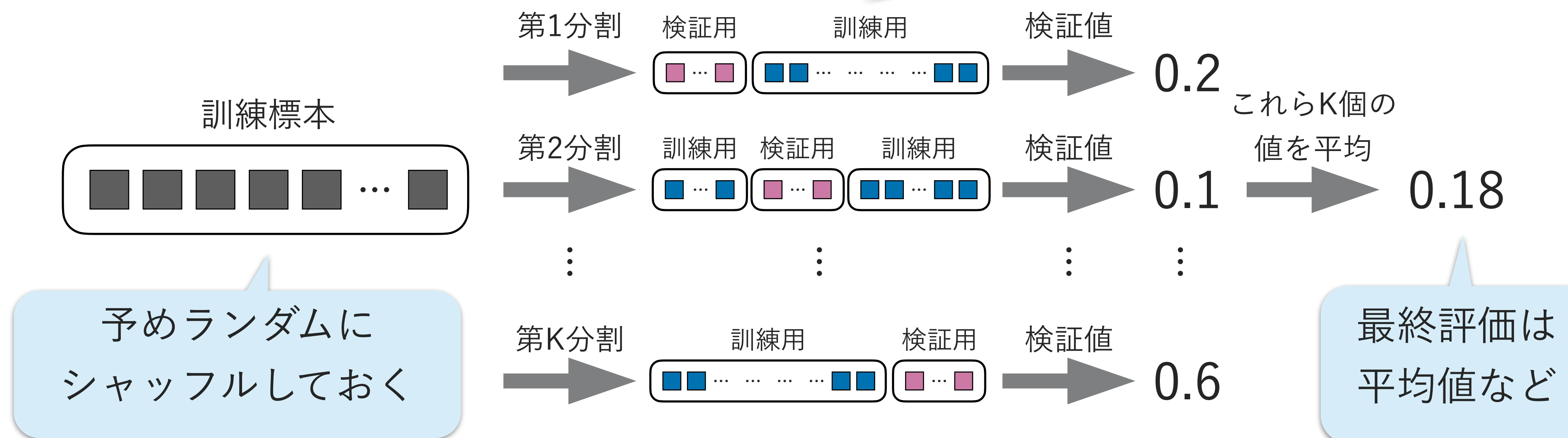
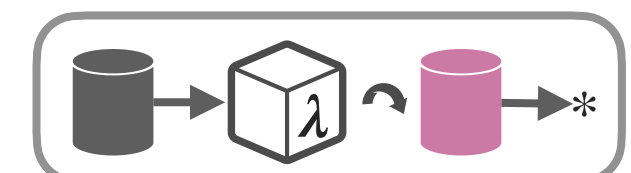
モデル選択規準 > K分割交差検証法

データが少ない場合の工夫

■ K分割交差検証 (K-fold cross validation)

取り置き法の分割を複数パターン行い，最終的には性能の平均などを用いる

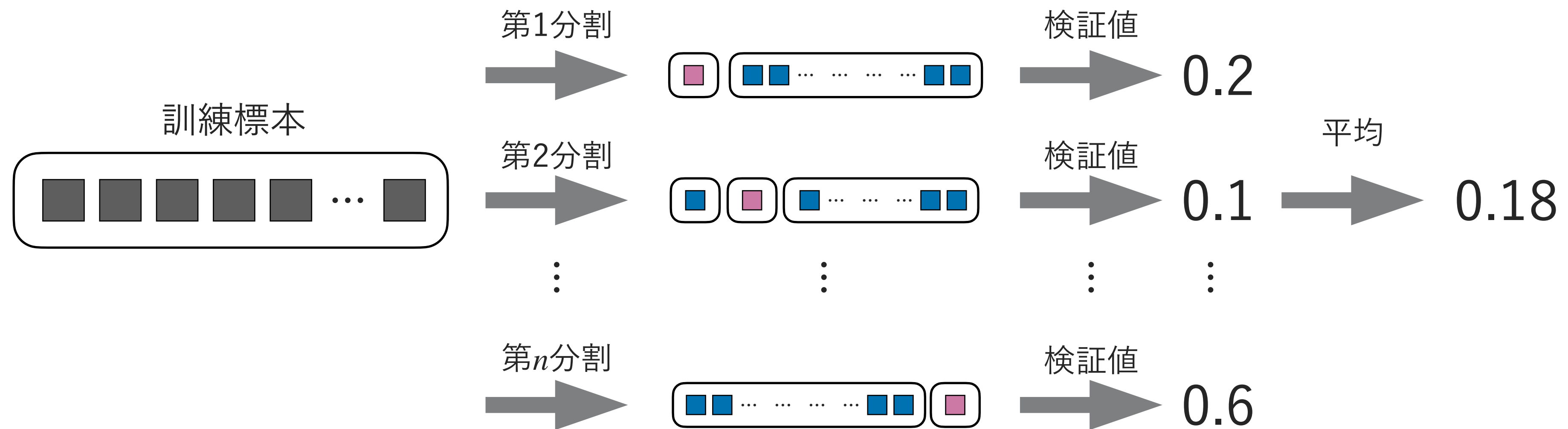
訓練用で学習，検証用で評価



モデル選択規準 > 1つ抜き交差検証法

検証用データサイズを1にして交差検証（つまり n 分割交差検証）

■ 1つ抜き交差検証（Leave-one-out CV, LOOCV）



【補足】 この場合， どの順番で検証用になるかは関係ないので， シャッフルしてもしなくても結果は同じになる。

モデル選択規準＞その他の検証・交差検証法

ケースバイケースで「適切な評価方法」が工夫される

■ 大原則は「最終的な性能の近似としてなるべく良い方法」を検証に使うこと

■ 他の分割方法

- 層化K分割交差検証（stratified K-fold CV）：クラス比率を保つように分割
- Kグループ抜き交差検証（leave-K-groups-out CV）：元々データにある何らかの集団の単位でデータを分割

モデル選択規準 > その他のモデル選択規準

一部のリスク関数や問題設定では、それに特化したモデル選択規準が開発

■ 情報量規準 (information criteria)

- AIC, BIC, WAIC, WBIC, TIC, ……といったものがある
- 基本的に、特定の種類のリスク関数（など）に対して導出される
- 例）AICは、KLダイバージェンスをリスクとする場合のモデル選択規準

■ 情報量規準では、訓練後の訓練損失値を補正して使うという方針が特徴的

- 検証データを取り置く方法との対比で言えば、訓練データ上での性能値を再利用する手法となっている

【用語】 AICは元々、An Information Criterionの略称だったが、後に提案者の赤池先生の名前で呼ばれるようになった（AICという略称は同じ）。

【補足】 情報量規準の中には、ここまで説明した機械学習の枠組みとは異なる目標の定式化に対して開発されているものもある。

モデル選択規準

どのモデル選択規準を使えばよいか？

- 仮定や目的によって異なる
 - それぞれ導出の仮定や妥当性の根拠が異なるので、仮定や目的によって異なる
- 予測タスクの教師付き学習の場合、検証データの置き換え法の汎用性が高い
 - リスク以外の評価指標にも簡単に適用しやすい（補助的な指標の併用など）
- 近年では……
 - データ量が少なければK-fold CVかLOOCVを用い、
 - データ量が多ければ、検証データの置き換え法で済ませてしまうことが多い

【補足】 検証データでの評価値という単純かつ妥当な方法が使えるのは、予測タスクの教師付き学習の場合だからである。他のタスクや学習戦略になると、話はそう単純ではない。

ハイパーパラメータ選択



ここまでで説明した，いずれかのモデル選択規準の値の大小で選ばよい

- 言い換えれば，ハイパーパラメータ選択という最適化問題を解けばよい

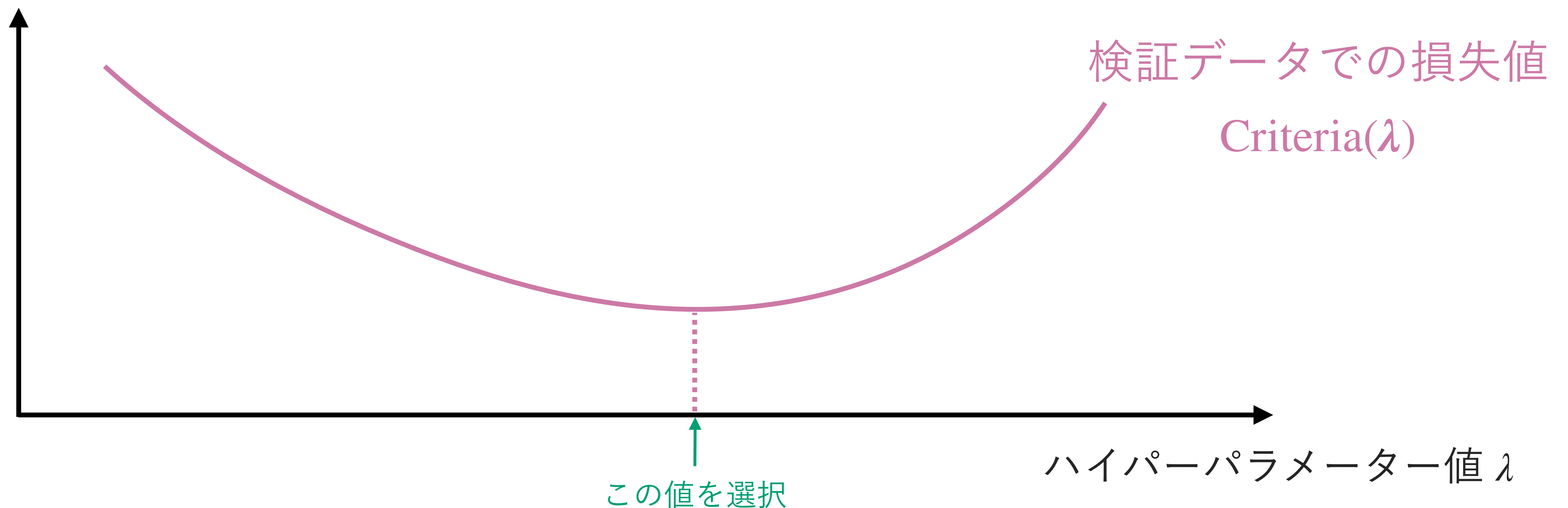
$$\underset{\lambda \in \Lambda}{\text{Min Criterion}}(\lambda)$$

- ここで $\text{Criterion}(\lambda)$ はハイパーパラメータ λ を用いて学習～検証を行なって得られるモデル選択規準の値．ここでは，小さいほど良い指標とする．
- $\text{Criterion}(\lambda)$ は，1つの λ に対して値を計算するために1回学習～検証をするので，大きな計算時間と計算費用がかかる傾向にある．

モデル選択 > 検証曲線 (Validation Curve)

ハイパーパラメーターを1つ選ぶだけならば，色々試してプロットすればよい

- **検証曲線**：ハイパーパラメーターの値に対して検証損失値をプロットしたもの

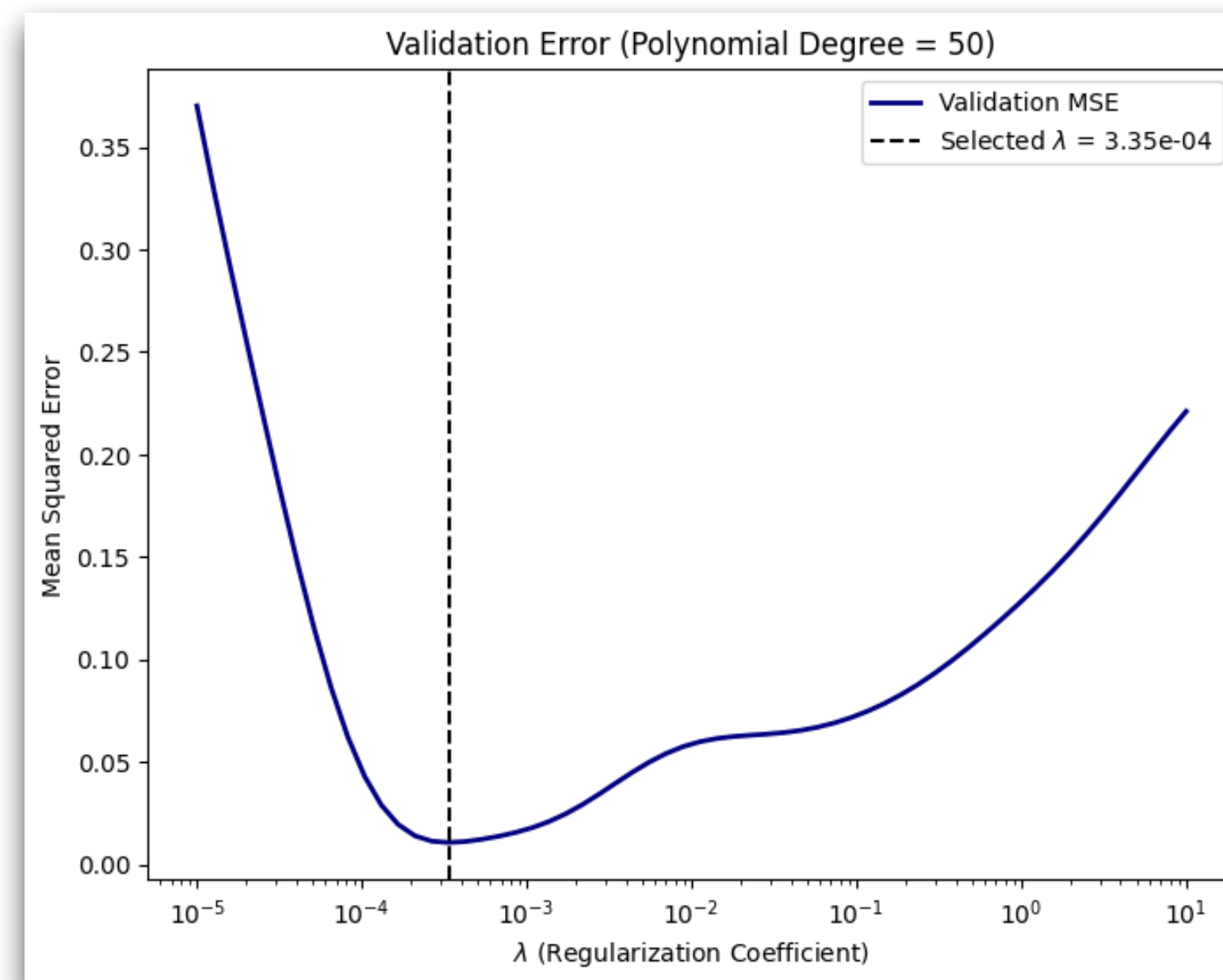


【補足】 もちろん，他のモデル選択規準を用いる場合は，その指標値をプロットして同様のことをすればよい。

モデル選択 > 検証曲線 (Validation Curve)

ハイパーパラメーターを1つ選ぶだけならば、色々試してプロットすればよい

- 実験 (50次多項式モデルによるL2正則化付き最小二乗回帰)
 - ハイパーパラメーター：正則化係数 λ ($\lambda \in \{10^{-5}, 10^{-4.9}, \dots, 10^{0.9}, 10^1\}$ を探索)

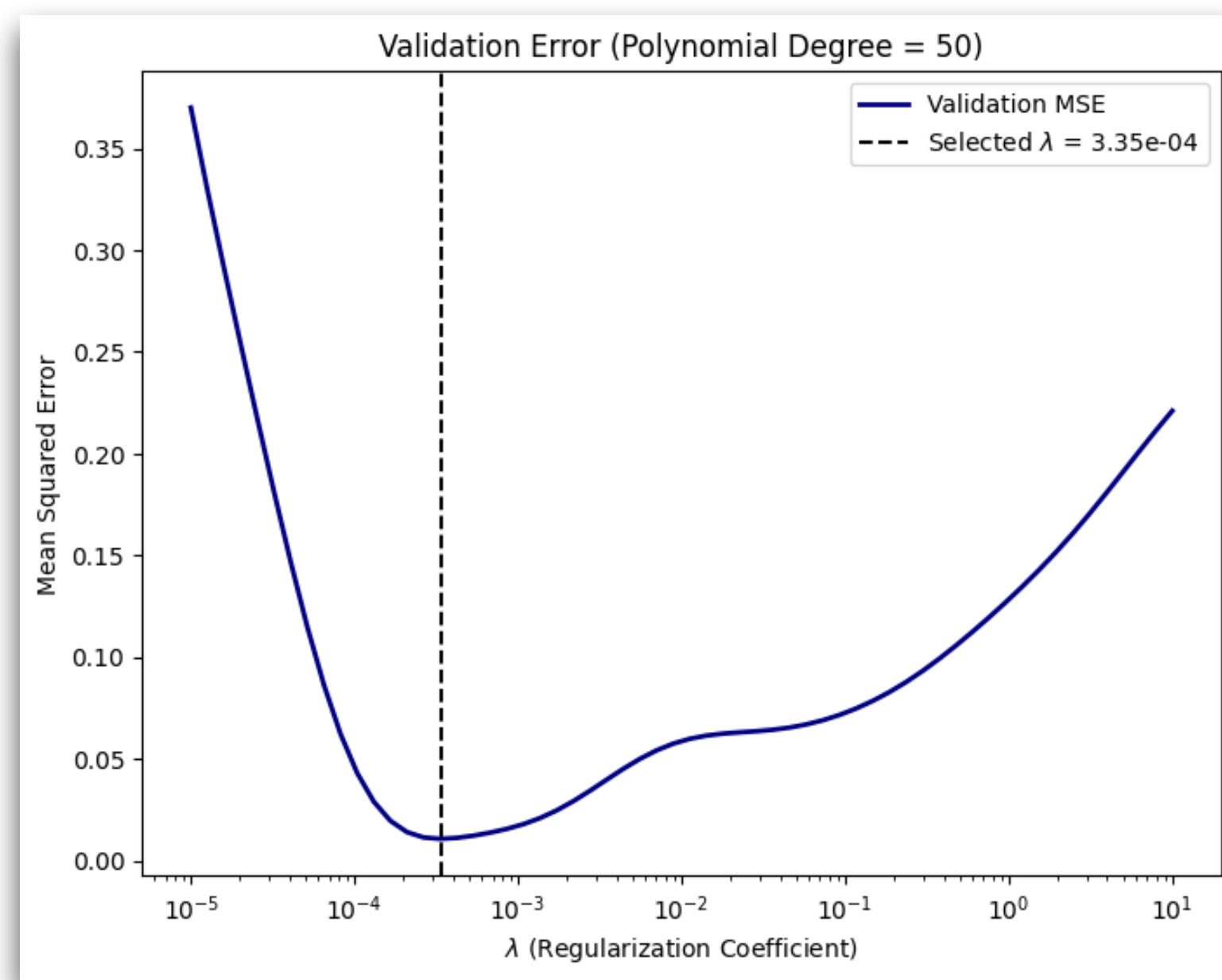


検証曲線

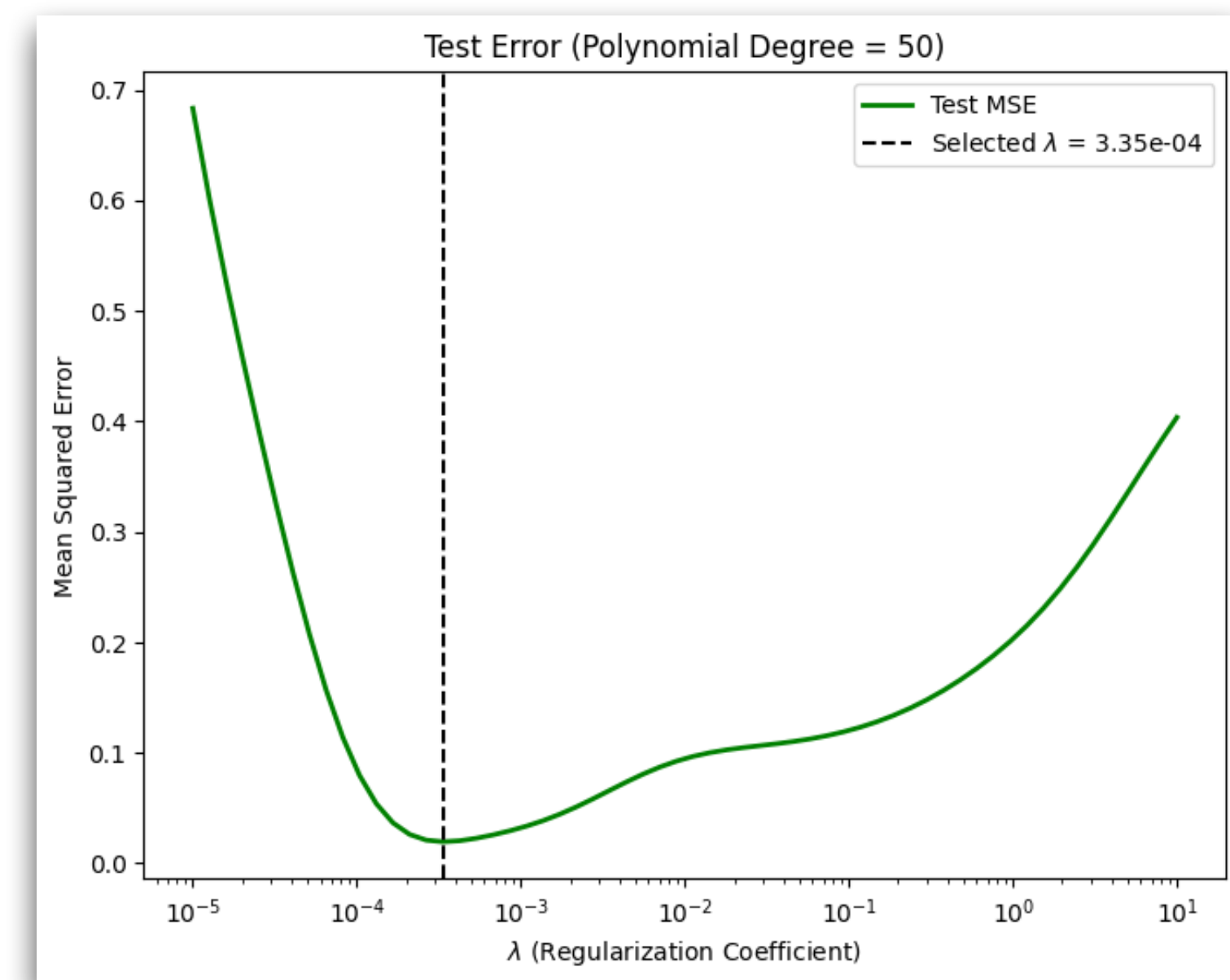
モデル選択 > 検証曲線 (Validation Curve)

ハイパーパラメーターを1つ選ぶだけならば、色々試してプロットすればよい

- 検証損失を用いて正則化係数を選択した結果、未知データでよく予測できている



検証曲線

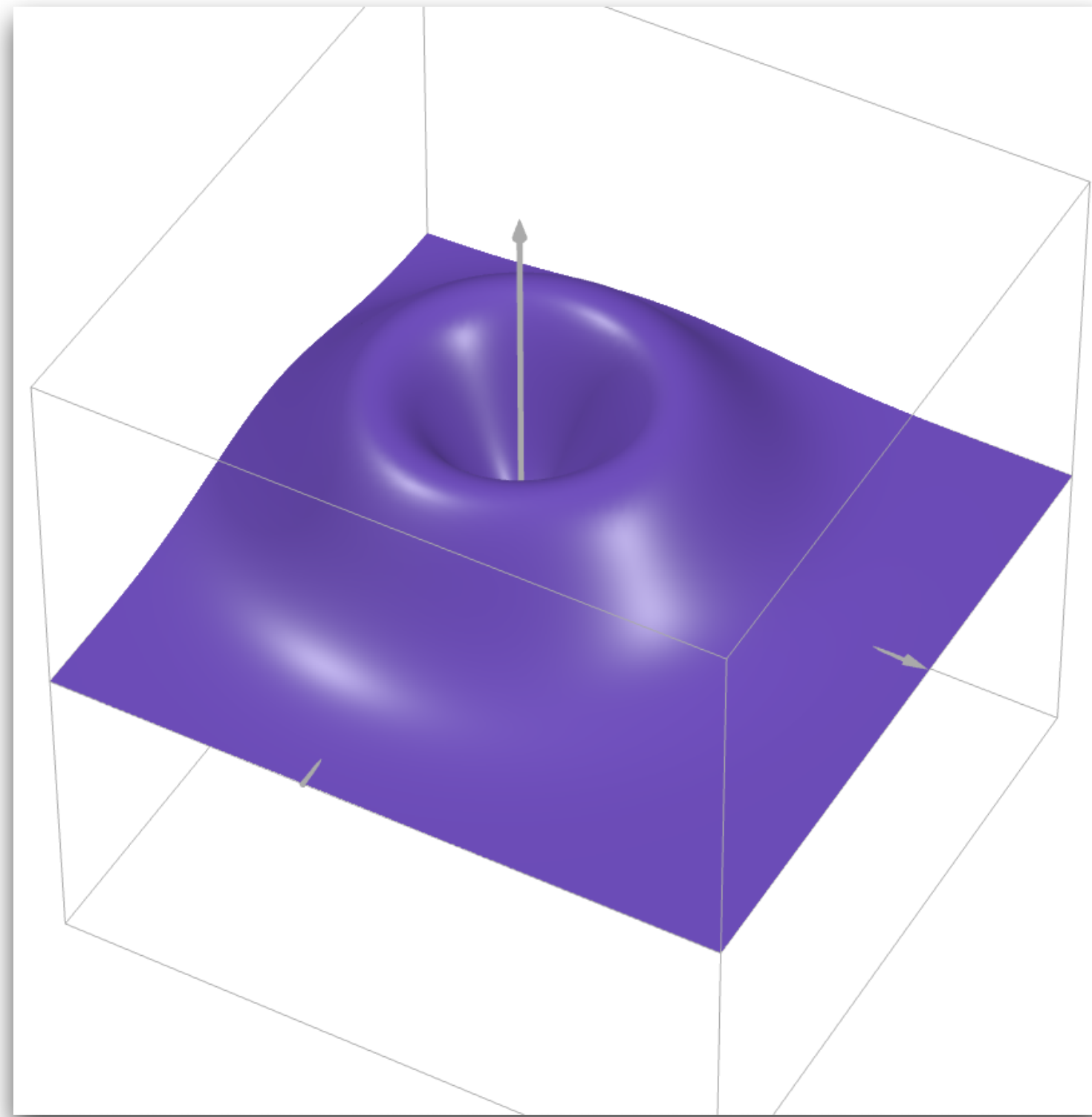


未知データでの評価値
(リスク)

モデル選択＞複数ハイパーパラメーターの最適化

ハイパーパラメーターが複数ある場合、性能は組合せで決まる

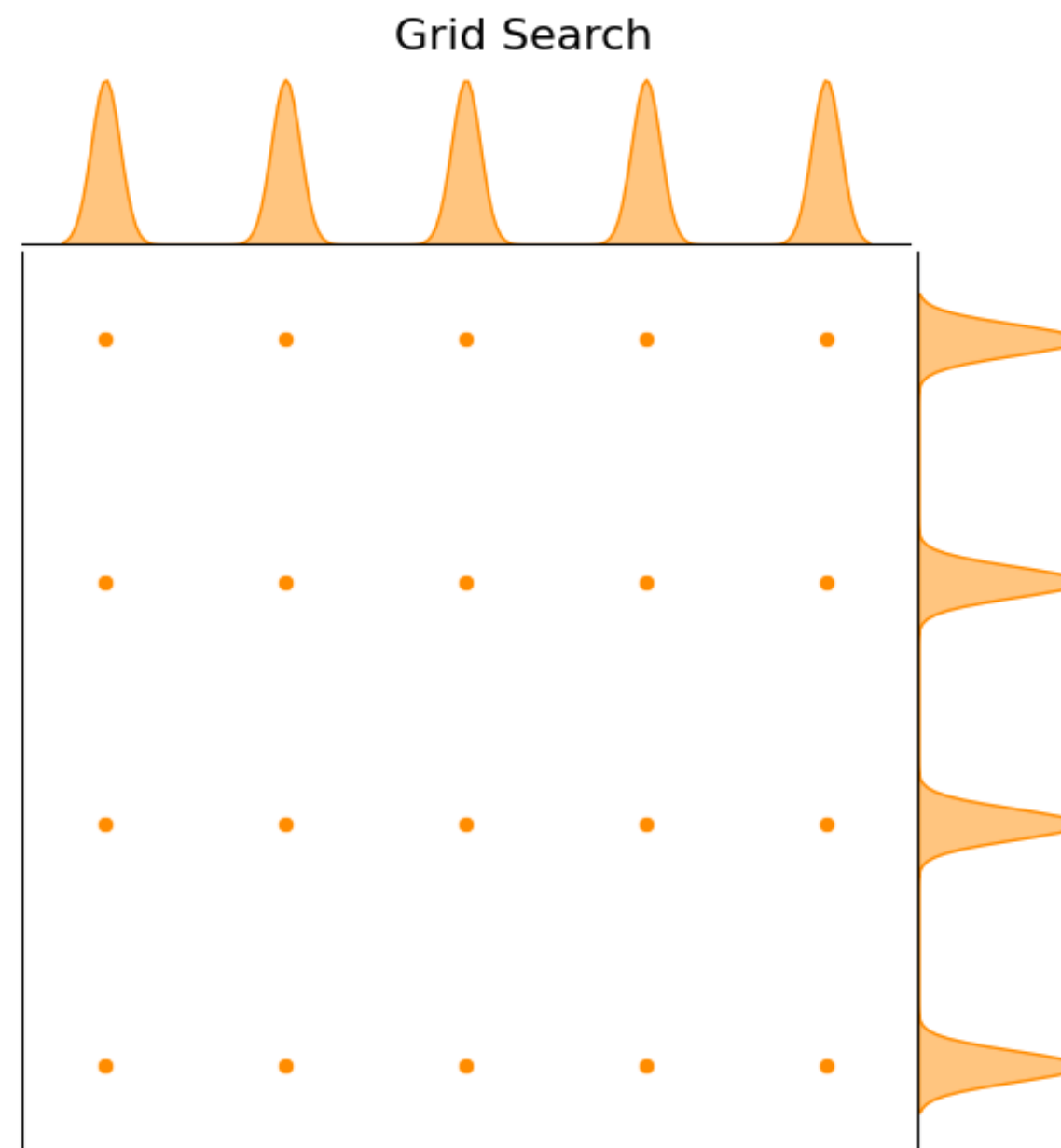
- ハイパーパラメーターが複数ある場合、組合せとして最適なものを選びたい



モデル選択 > グリッドサーチ

多数のハイパーパラメーターを，愚直に総当たりで試すのは得策ではない

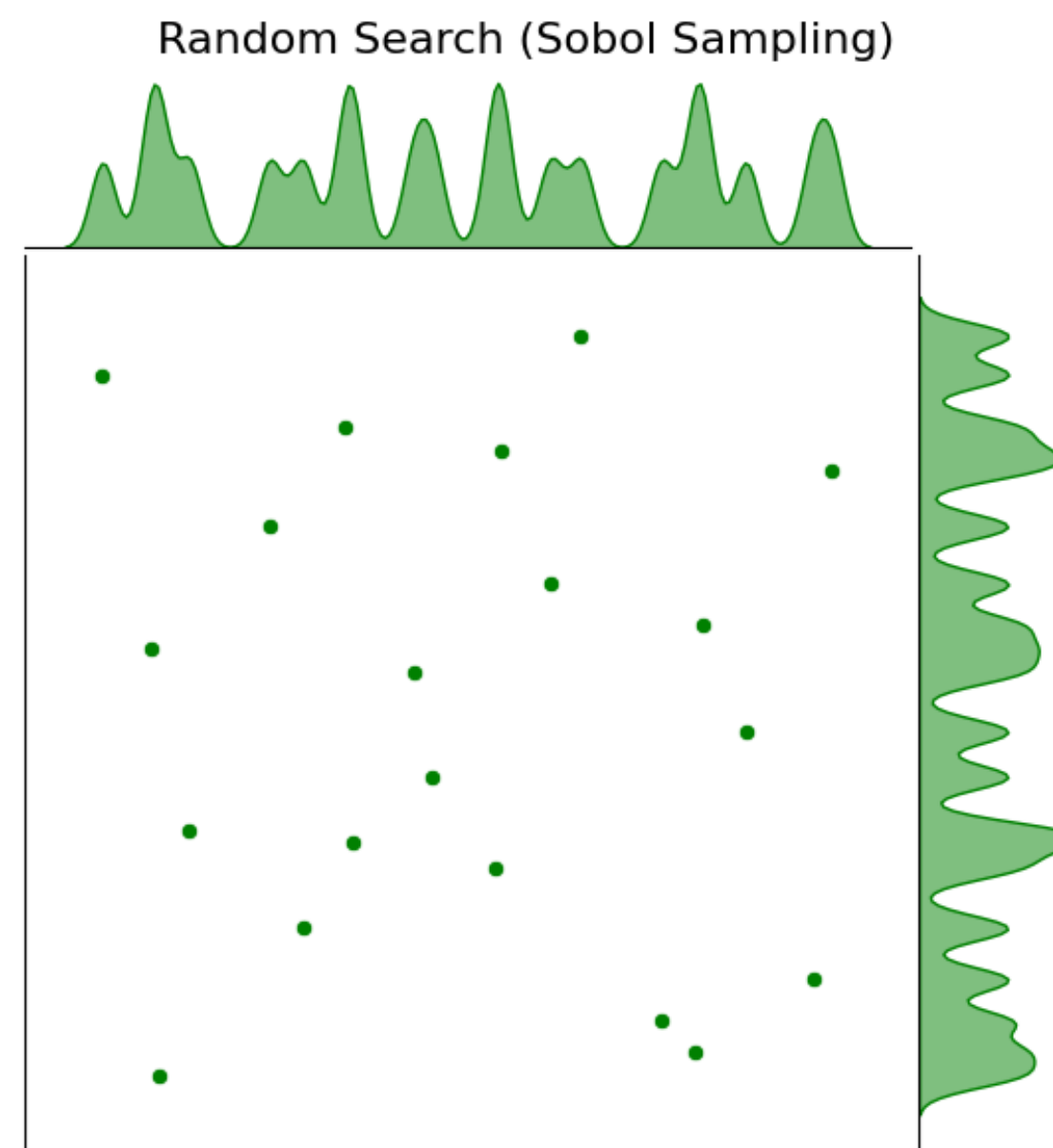
- グリッドサーチ（grid search）：全ての組み合わせを総当たりする方法
 - 少数なら実行可能だが，例えば5つのハイパーパラメーターを各8水準から探索するなら， $8^5 = 32,768$ 回の学習～評価を繰り返すことになり非現実的



モデル選択 > ランダムサーチ

総当たりできない場合のための様々な方法が研究されている

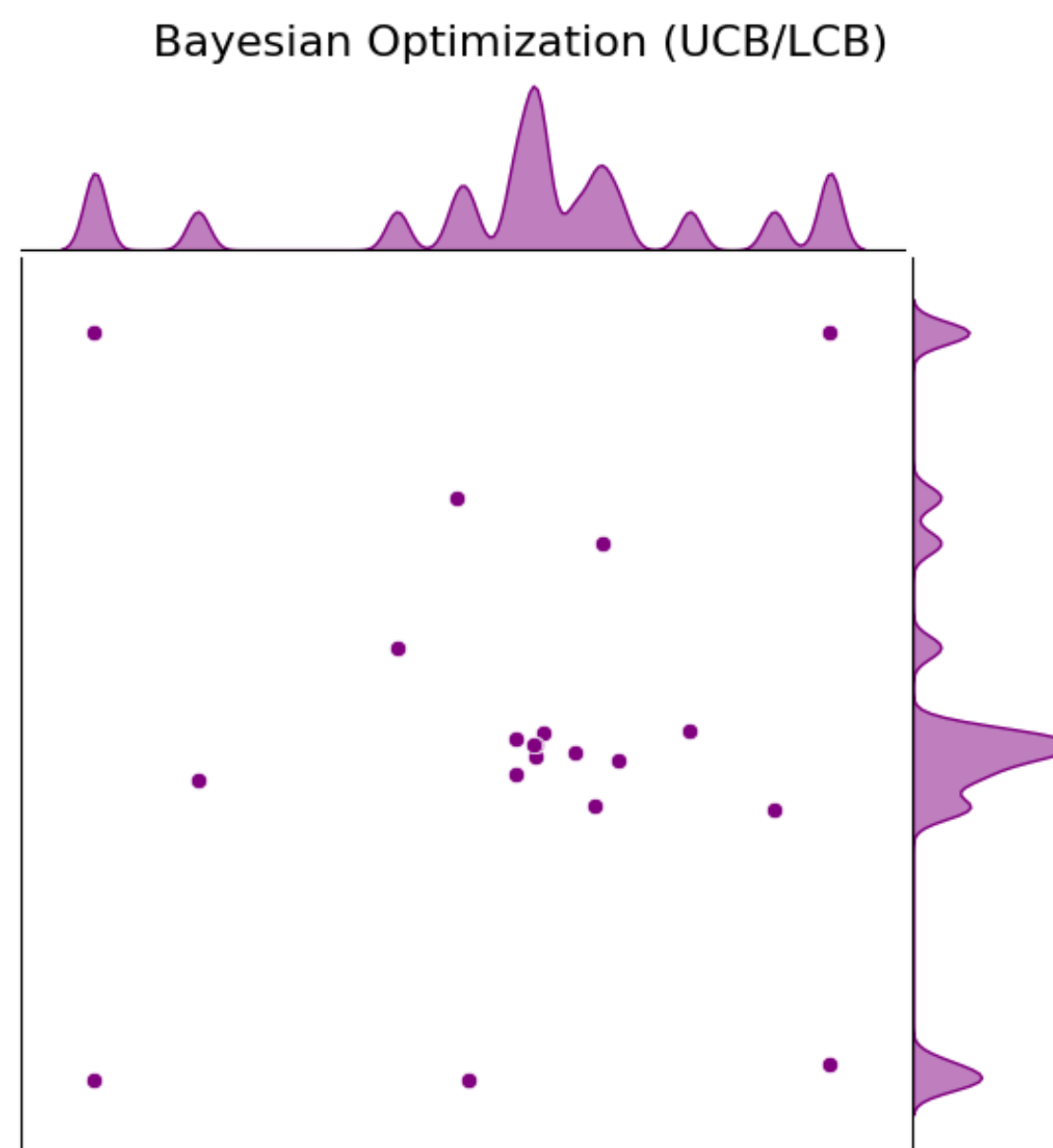
- ランダムサーチ (random search)
 - 何らかの確率分布でハイパーパラメーターをランダムにサンプリング
 - グリッドサーチと比べて満遍なく全体をカバーしている



モデル選択 > ベイズ最適化

総当たりできない場合のための様々な方法が研究されている

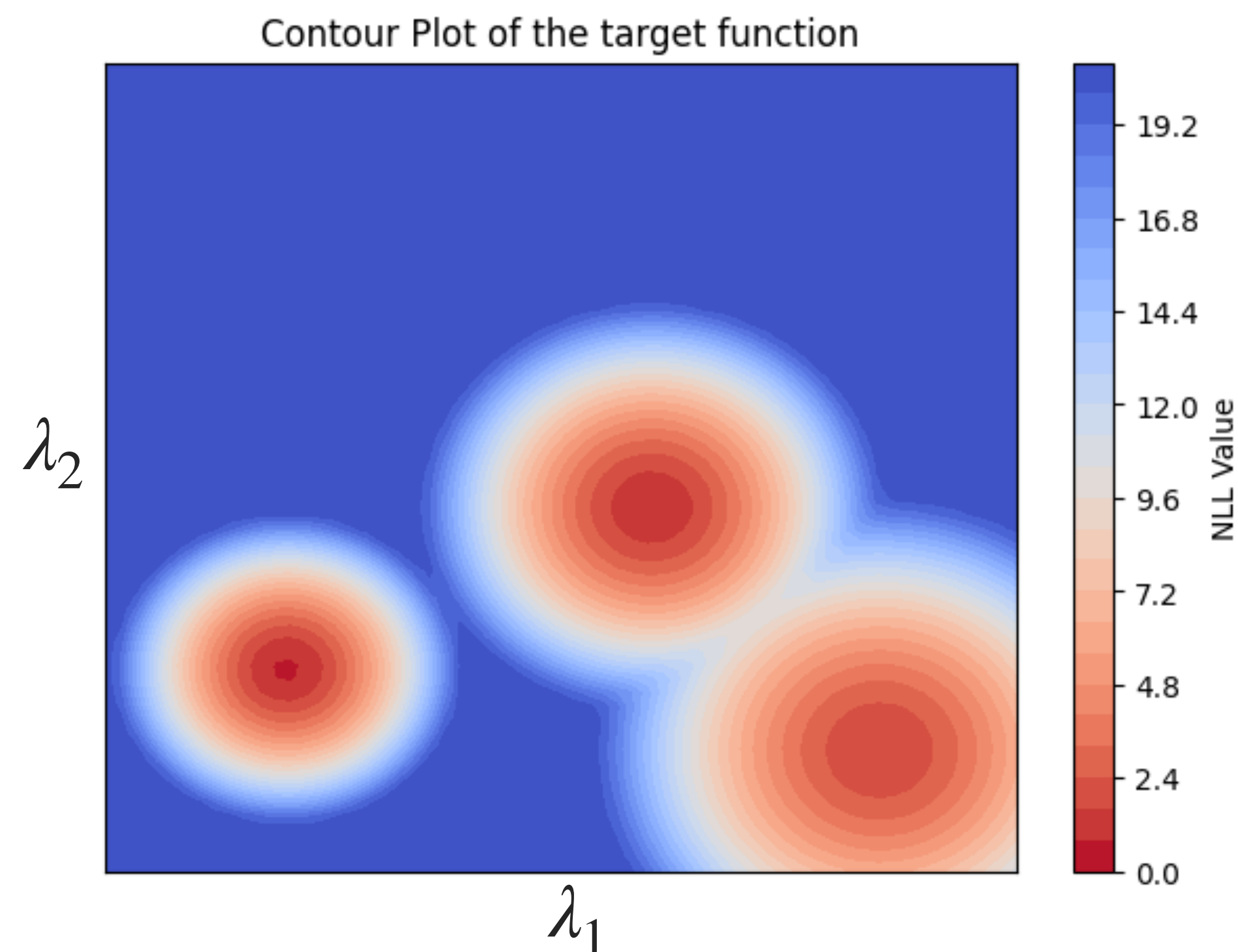
- ベイズ最適化 (Bayesian optimization)
 - モデル選択規準の値を目的関数として、最適化問題を解く方法
 - 計算済みの目的関数値の情報から、当たりをつけながら逐次的に探索する



モデル選択 > ハイパーパラメーター探索の実験

以下の関数を最小にする (λ_1, λ_2) を探索

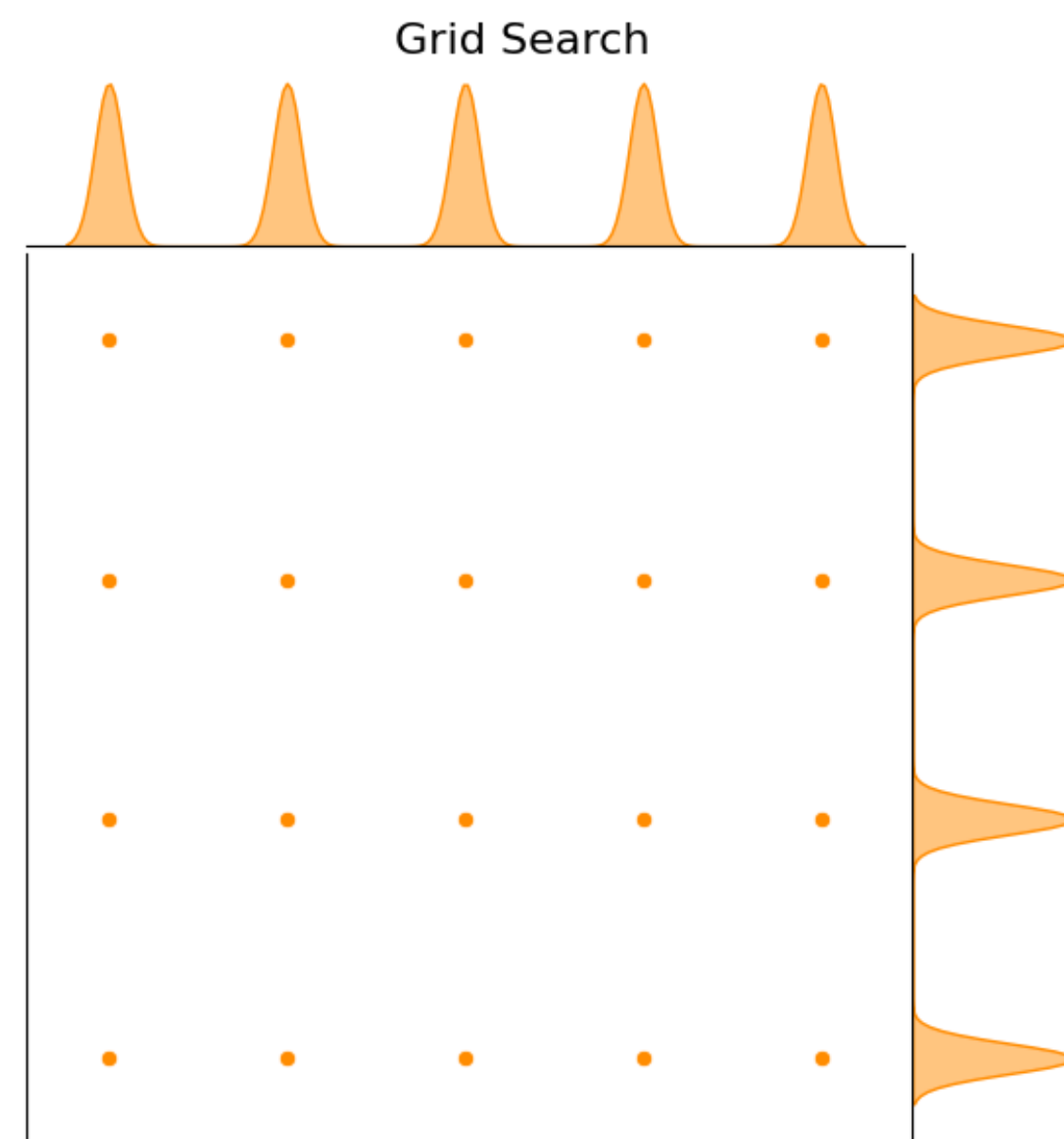
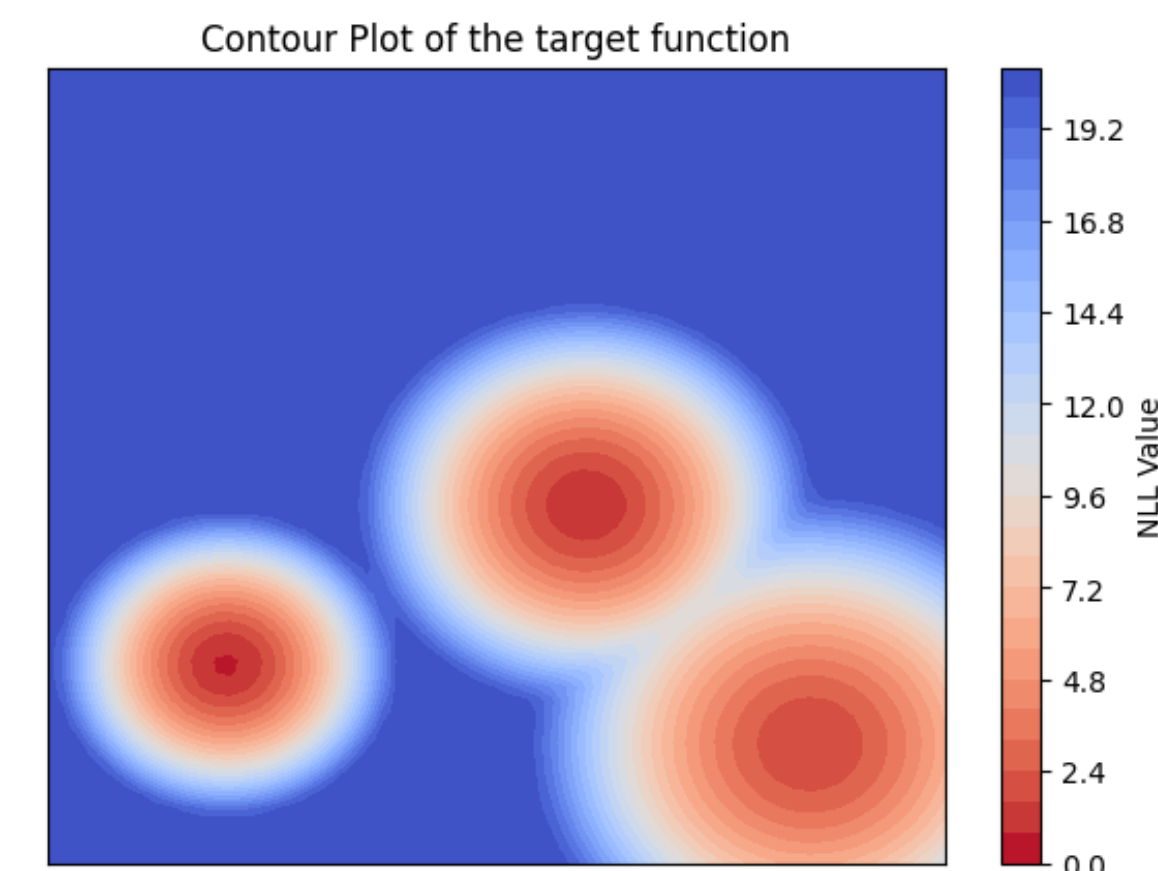
- 関数 $\text{Criterion}(\lambda_1, \lambda_2)$ (値が小さいほど良い, 赤色で表示)
 - 少ない評価回数で, 効率よくハイパーパラメーター選択をしたい



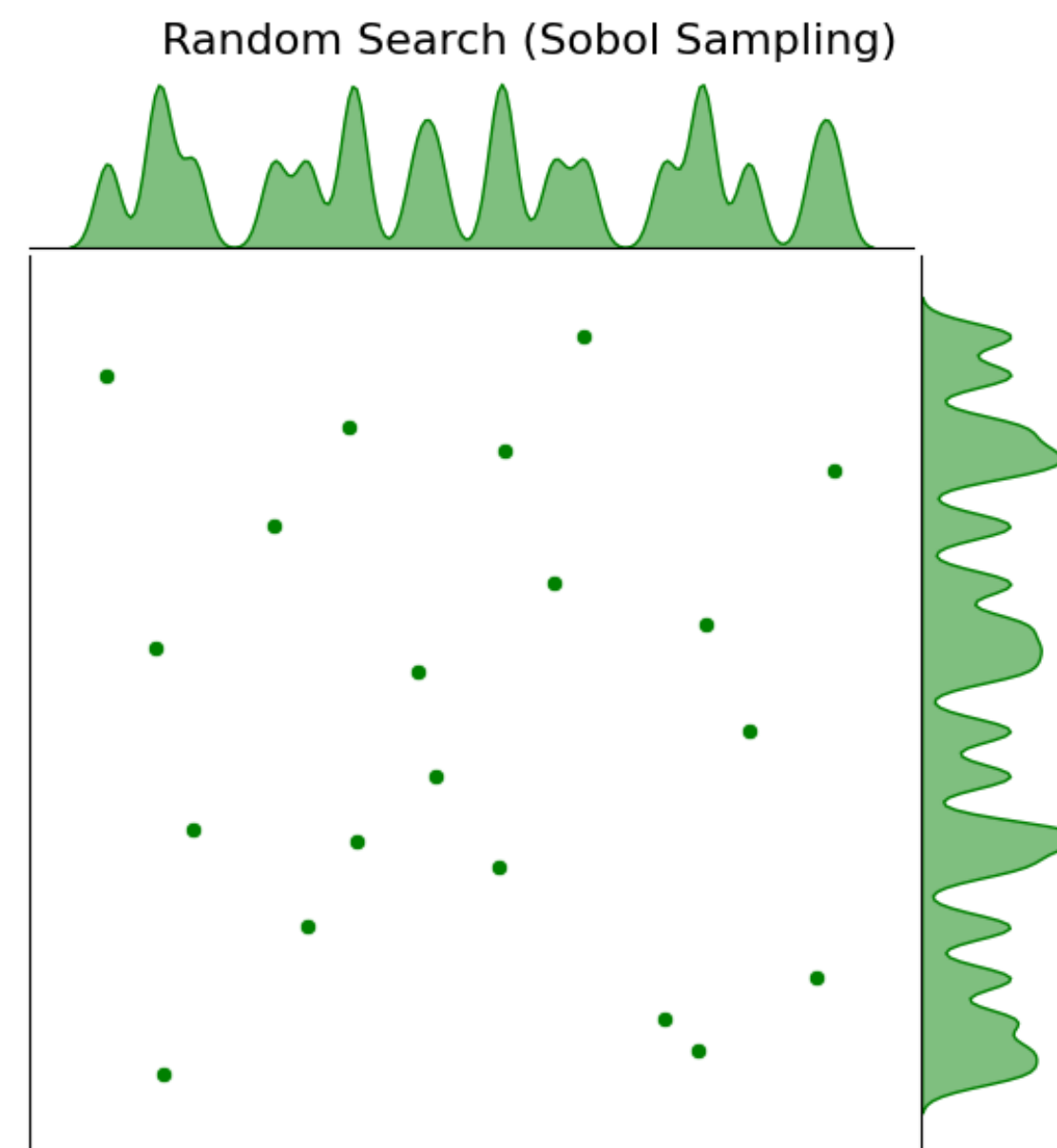
モデル選択 > 実験結果

ランダムサーチやベイズ最適化を用いることで高効率に

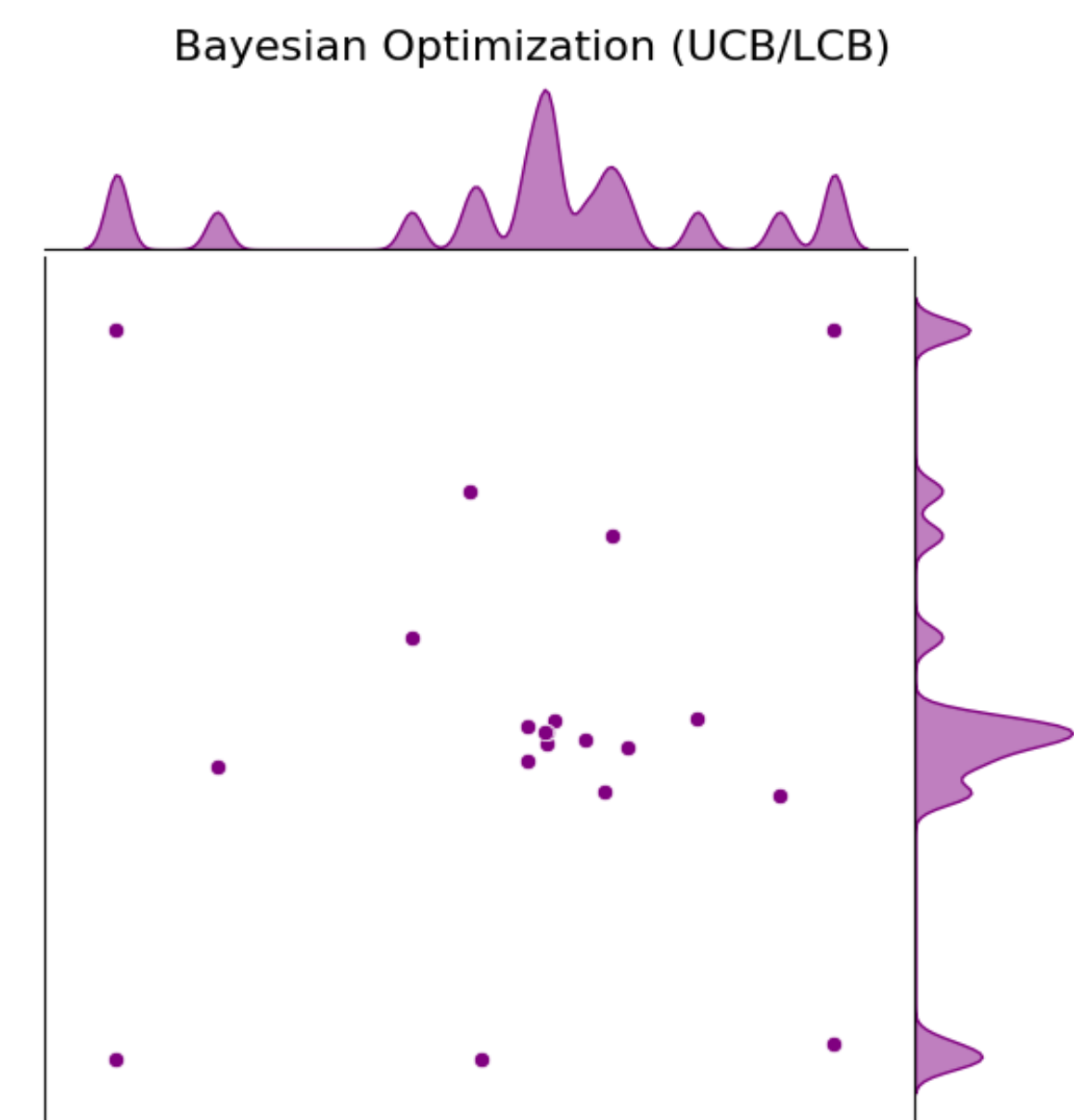
- 手軽なのはランダムサーチ（並列化しやすい）
- 賢く実行する必要があるらベイズ最適化



グリッドサーチ：5.9848



ランダムサーチ：2.9773



ベイズ最適化：0.9219

ロードマップ

